Methodology and Extended Numerics for Pruning Deep Neural Networks via the Marchenko–Pastur Distribution

Companion methodology paper to the main manuscript

Leonid Berlyand
Department of Mathematics, Pennsylvania State University

Theo Bourdais
Department of Computing and Mathematical Sciences, Caltech

Houman Owhadi
Department of Computing and Mathematical Sciences, Caltech

Yitzchak Shmalo*
Department of Mathematics, Pennsylvania State University

Abstract

This document is the companion methodology and extended-numerics paper to the manuscript *Pruning Deep Neural Networks via the Marchenko–Pastur Distribution* (henceforth Main). Main contains the theoretical content (deterministic pruning certificates, Gaussian/sub-Gaussian random-matrix specializations, full proofs) and the headline empirical tables. This document, which lives at https://github.com/yspennstate/RMT_based_pruning_in_deep_learning alongside the code that produced every number in Main, contains everything the rigorous reader needs in order to reproduce the experiments: full repository architecture, the projection and fine-tuning code paths, the speedup-measurement protocols, and the migrated extended-numerics appendices that previously inflated Main. The reader should consult this paper whenever Main cites Section M.X — those citations are pointers into this document.

Companion repository. https://github.com/yspennstate/RMT_based_pruning_in_deep_learning

17-model checkpoint archive (Hybrid Magnitude–SER). https://drive.google.com/drive/folders/1mm990SHAHlYdISHxirvMRdVQEAjpIxDd

Cross-paper conventions. We refer to results in Main as Main-Theorem X.Y, Main-Table Z, etc. Norm conventions, theorem numbering, and notation are inherited from Main.

Contents

*Corresponding author: Yitzchak Shmalo. Email: yitzchak.shmalo@gmail.com.

1 Introduction and how to read this paper

1.1 Scope

MAIN establishes a Marchenko–Pastur (MP)-driven theory of deterministic pruning certificates and reports headline numerical results: the multi-architecture Hybrid Magnitude–SER table, the CAST 2:4 + ToMe accuracy/throughput row, and the deterministic-certificate audit. It is necessarily a long document because it contains the proofs. This methodology paper carries everything that is operational rather than theoretical:

1. the full RMT-pruning protocol stack (BEMA, Spectral Edge Budgeting, Spectral Edge Reallocation, Hybrid Magnitude–SER);
2. the CAST and CAST-2E pipelines (cert-aware k:n projection, Cin permutation alignment, free restoration, frozen-mask distillation);
3. all hardware speedup measurements, including the A100 native-2:4 throughput data and the 6:12 $\rightarrow$ 2:4 projection benchmark used to translate non-deployable cert-optimal cells into deployable ones;

4. the certification-audit numerics (three bridges from MAIN-§5: top-1 vs. B_T, Lemma 5.4 violations, $\sigma_+ \to B_{\text{proxy}}$ within-cycle correlations);

5. the extended fully-connected MP-pruning experiments;

6. ledgered checkpoint validation tables.

1.2 When to read which paper

MAIN alone is sufficient for: the theoretical statements, all proofs, the headline accuracy + speedup numbers (Table 2 and the CAST FLOP table), the theory-to-numerics mapping table, and the high-level reproducibility narrative.

This paper is sufficient for: replicating any individual experiment, understanding the relationship between every line of code in the GitHub repository and the numerical claims in MAIN, and the full extended-numerics record.

1.3 Repository at a glance

The companion GitHub repository contains, at the time of writing:

- Approximately 60 Python scripts at the repository root implementing the classical RMT, magnitude, Hybrid Magnitude–SER, four-regime SEB, and drop-threshold protocols (these underlie MAIN-§3 and the multi-architecture validation table);
- A self-contained `cast_2e/` subtree containing the new CAST-2E pipeline: certificate-aware k:n projection (with $k = 2, 4, 6, 8, 12$ and $n = 4, 8, 12, 16$ supported), Cin permutation alignment, in-place inline fine-tuning runners avoiding the perm-hook serialization issue, and the suite of speedup benchmarks;
- Eighteen sweep-result JSONs covering all three architecture families (ResNet50, ConvNeXtV2-Base, ViT-B/L), six k:n patterns, dense vs. SER source ablations, and α_{ser} sweeps;
- Five A100 throughput JSONs (dense vs. 2:4 vs. ToMe-only vs. 2:4+ToMe) plus the 6:12 $\to$ 2:4 projection-speedup data;
- All post-fine-tuning evaluation JSONs underlying the new headline FLOP table in MAIN.

2 Repository tree and architecture

2.1 Top-level layout

Listing 1: Top-level repository layout.

```
RMT_based_pruning_in_deep_learning/
+- README.md # repo overview, quick start
+- LICENSE # MIT
+- requirements.txt # canonical deps
+- adaptive_rmt/ # RMT diagnostics: BEMA, alpha
+- configs/ # YAML/JSON sweep configs
+- model_queue_runs/ # cached SVD/ESD per model
+- scripts/ # shell launchers / watchers
+- src/ # shared utilities
+- prune.py # the pruning entrypoint
+- fine_tune.py # the fine-tuning entrypoint
```

```
+- magnitude_rmt_sweep.py # the full sweep driver
+- rmt_magnitude_sweep.py # alt sweep ordering
+- hybrid_mag20_then_v8*.py # Hybrid Magnitude--SER
+- run_finetune_*.py # the FT runners (4 variants)
+- haar_*.py # Haar-SV preprocessing
+- iterative_*.py # iterative growing-a + 5pct
+- pruning_method_comparison_sweep.py # cross-method comparison
+- run_removed_matrix_audit_v*.py # certificate audit drivers
+- analyze_sweep.py # JSON -> table aggregator
+- direct_run_watchdog.py # crash-resilient runner
+- queue_*.py # multi-run queue managers
+- remote_*.py # remote pod helpers
+- cast_2e/ # NEW (this paper's content)
   +- methodology.tex # this document
   +- code/ # cert-aware k:n + FT runners
   +- scripts/ # launch / watcher scripts
   +- benchmarks/ # A100/L4 throughput JSONs
   +- benchmarks_speedup/ # 6:12->2:4 speedup JSONs
   +- masks_archive/ # per-layer mask stats
   +- post_ft_eval/ # post-FT eval JSONs
   +- sweep_results/ # k:n sweep cell JSONs
   +- AUDIT.md # full data-flow audit
   +- THEORY.md # CAST-2E theory map
```

2.2 How to navigate by paper section

Table 1 provides the canonical pointer from each MAIN section/result to the code path that produces it. This is the table to keep open while reading MAIN.

2.3 Coding conventions in the repository

The classical RMT, magnitude, and Hybrid Magnitude–SER scripts share a common contract:

- Each entrypoint reads a YAML/JSON config from `configs/` and writes its results to a per-run subdirectory of `model_queue_runs/<arch>/<config_id>/`.
- Every fine-tune call emits both a final `finetune_results.json` and a per-epoch ledger that the analyzer can consume even if a run is killed mid-epoch.
- The SVD and BEMA edge fits are cached in `model_queue_runs/<arch>/svd_cache/` so that re-running a sweep with a different sparsity schedule does not pay the SVD cost.

The CAST-2E scripts in `cast_2e/code/` share a different but equally explicit contract:

- All projection scripts (`project_*.py`) emit a `student_pre_ft.pt` payload containing both the projected state dict and a JSON-serializable `cert_config` dictionary.
- The fine-tuning runners are deliberately "inline": they receive the calibration loader and projection arguments directly, run projection in-process, and then immediately enter the distillation loop with the projected weights and frozen mask, without an intervening save/load round-trip. This avoids a perm-hook serialization issue described in §3.

Table 1: Mapping from MAIN sections and results to code paths in the GitHub repository. "[...]" refers to wildcards covered by the listed file or by closely related variants in the same directory.

MAIN location	Code	METHODOLOGY §
MAIN-Table 2 (multi-arch Hybrid Mag–SER)	`hybrid_mag20_then_v8*.py`, `run_finetune_magnitude_v3.py`, `prune.py`	§8
MAIN-Table CAST FLOPs (paper-headline FLOP table)	`cast_2e/code/project_cast_sparsity.py`, `cast_2e/code/run_vitb_ft_inline.py`, `cast_2e/code/run_resnet_ft_inline.py`	§8
MAIN-§3 RMT-guided pruning	`magnitude_rmt_sweep.py`, `pruning_method_comparison_sweep.py`	§12
MAIN-§5.4 deterministic certificate (§5.4 audit)	`run_removed_matrix_audit_v8_model_exec.py`	§11
SEB-budget hyperparameter sweeps	`configs/seb_*.yaml`, `magnitude_rmt_sweep.py`, `rmt_magnitude_sweep.py`	§9
BEMA edge fitting	`adaptive_rmt/bema.py` (and equivalents)	§8
α power-law signal	`adaptive_rmt/alpha.py`	§9.the main paper
	`haar_optuna*.py`	
A100 throughput benchmark (2:4)	`cast_2e/code/benchmark_sparse_throughput.py`	§5
6:12 → 2:4 projection speedup	`cast_2e/code/benchmark_6_12_to_2_4_projection.py`	§5
k:n sweep driver (sweeps in this paper)	`cast_2e/code/cert_opt_eval_kn*.py`	§6
ViT-B inline FT (paper headline 83.74)	`cast_2e/code/run_vitb_ft_inline.py`	§4
ResNet inline FT (paper headlines 75.67, 78.00, 80.59, 81.33)	`cast_2e/code/run_resnet_ft_inline.py`	§4
ConvNeXtV2 D1216/D816 (paper headlines 86.35, 85.85)	`cast_2e/code/project_convnextv2_d1216.py` + ToMe runner	§4

3 Pre-fine-tuning projection: cert-aware k:n structured sparsity

This section documents the pre-FT projection family generalising CAST (MAIN-Appendix referenced as Section M.10) from 2:4 to general k:n. The 2:4 design and Stage-1/Stage-2 split are the same as in CAST; what is new is

1. parametric k:n at the row-group level, with $n \in \{4, 8, 12, 16, 32\}$ and $k \in \{1, \dots, n-1\}$;
2. Cin permutation alignment within each layer's input groups, so that the cert-cost minimisation can choose patterns from a larger set of feasible k:n blocks;
3. an SER-prior term (the α_{ser} term of §3.3) that anchors the chosen pattern to the Hybrid Magnitude–SER mask;
4. native support for both Linear and Conv2d layers, with the convolutional case implemented by unfolding the kernel.

3.1 Generalised cost: k:n row-group search with free restoration

Fix a Linear layer $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ with $d_{\text{in}} = nG_\ell$. Reshape rows into contiguous n-tuples: $W_{i,g,k}$ is the k-th weight of group g in output row i, $i \in [d_{\text{out}}]$, $g \in [G_\ell]$, $k \in [n]$.

For each pair (i, g) the k:n search enumerates the $\binom{n}{k}$ keep patterns

$$\mathcal{M}_{k:n} = \{m \in \{0,1\}^n : |m|_0 = k\}.$$

For $n = 4, k = 2$ this is the 6 patterns of CAST (MAIN-§G); for $n = 16, k = 8$ it is $\binom{16}{8} = 12870$ patterns. When the enumeration becomes prohibitive, `project_kn_sampled.py` provides a sampled variant.

The materialised slot value retains CAST's three-case rule (MAIN-Eq. G.1):

$$v_{i,g,k}(m) = m_k \cdot \begin{cases} W^{\mathrm{SER}}_{i,g,k}, & M^{\mathrm{SER}}_{i,g,k} = 1, \\ W^{\mathrm{dense}}_{i,g,k}, & M^{\mathrm{SER}}_{i,g,k} = 0 \text{ and free-restoration enabled}, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

The "dense" source corresponds to setting $W^{\mathrm{SER}} \equiv W^{\mathrm{dense}}$ and $M^{\mathrm{SER}} \equiv 0$, in which case Eq. (1) reduces to $v_{i,g,k}(m) = m_k W^{\mathrm{dense}}_{i,g,k}$. We use "dense source" for ConvNeXtV2 because the SER ckpts at $s = 0.35$ for that model lose more accuracy at projection than the dense ckpt does; for ResNet50 the SER source helps; for ViT-B the SER source helps with $\alpha_{\mathrm{ser}} = 0.5$.

The within-group calibration covariance and cert-inspired cost are inherited from MAIN-Eqs. G.2–G.3:

$$C_g = \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} h_g(x) h_g(x)^\top, \qquad c_{i,g}(m) = r_{i,g}(m)^\top C_g \, r_{i,g}(m), \qquad r_{i,g}(m) = (1 - m) \odot v_{i,g,:}(m). \tag{2}$$

The argmin is solved by enumeration; tensor-shape $(\binom{n}{k}, d_{\mathrm{out}}, G_\ell)$. For $\binom{n}{k} \geq 5{,}000$ the sampled variant draws $\binom{n}{k}_{\mathrm{eff}} = 1024$ candidates uniformly without replacement.

3.2 Cin permutation alignment ("flatten the layer")

The native $k{:}n$ search treats the n input slots within each group as fixed by the natural channel order. Cin-permutation aligns each layer's input columns by the importance score

$$s_j = \sum_i \left| W^{\mathrm{dense}}_{i,j} \right| \cdot |\bar{h_j}|, \qquad |\bar{h_j}| = \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} |h_j(x)|,$$

and then quartile-interleaves the columns: column j in the permuted layer is the $\pi(j)$-th most important one, where π is the importance-balanced permutation that places the g-th group's k columns into quartile positions $\big(g,\ g + G_\ell/4,\ g + 2G_\ell/4,\ g + 3G_\ell/4\big)$ in the original ordering. We call this "flattening the layer" because it makes each group span the full importance range, so that the cert-cost minimiser sees a maximally informative set of candidates within every group.

The permutation is implemented as a forward pre-hook, with the inverse permutation applied at the layer output to keep downstream activations aligned. The hook is registered at projection time and removed at the post-FT save step.

3.3 The α_{ser} Hamming-distance prior

To bias the per-row cert-cost minimiser toward the Hybrid Magnitude–SER mask without overriding it, we add a Hamming penalty (MAIN-Eq. G.4 generalised):

$$c^{\mathrm{prior}}_{i,g}(m) = \alpha_{\mathrm{ser}} \cdot \bar{c}_g \cdot \left| m - M^{\mathrm{SER}}_{i,g,:} \right|_1, \qquad \bar{c}_g = \mathrm{mean}_{i,k}\, v^2_{i,g,k} \cdot \mathbb{E}_x \big[h_{g,k}(x)^2 \big]. \tag{3}$$

Setting $\alpha_{\mathrm{ser}} = 0$ recovers pure cert-aware projection; $\alpha_{\mathrm{ser}} \to \infty$ forces the SER mask. We sweep $\alpha_{\mathrm{ser}} \in \{0, 0.1, 0.25, 0.5, 1.0, 2.0\}$ in the cell sweeps reported in §6; the empirical sweet spot is $\alpha_{\mathrm{ser}} = 0.5$ for ViT-B with the SER-source variant.

3.4 Conv2d generalisation

For a Conv2d kernel of shape $(C_{\text{out}}, C_{\text{in}}, k_h, k_w)$ we unfold the kernel into a $(C_{\text{out}},\ C_{\text{in}} \cdot k_h \cdot k_w)$ matrix and apply the same row-group search. The projection runs on 1×1 and 3×3 convolutions; depthwise convolutions and the first input-projection conv are excluded by an `is_eligible_conv` predicate exposed via `cast_2e/code/project_kn_sparsity.py`.

In ConvNeXtV2 the projection is restricted to Linear layers (the per-block depthwise 7×7 convolutions are excluded) because the depthwise structure interacts pathologically with the row-group $k{:}n$ pattern; this restriction is recorded in `cast_2e/AUDIT.md` as a known limitation.

3.5 Inline projection + FT runner (avoiding the perm-hook bug)

A subtle but load-bearing implementation issue: when a projection registers Cin-permutation forward pre-hooks on a model and we save the resulting state-dict to disk, the hooks are not part of the state-dict. A naive reload at FT time produces a model whose weights are in the permuted ordering but whose forward pass is unpermuted; on ImageNet val the reloaded model scores top-1 ≈ 0.0006 (random-class chance for a 1000-class problem). `cast_2e/code/run_vitb_ft_inline.py` and `cast_2e/code/run_resnet_ft_inline.py` avoid this by performing projection in-process, immediately followed by FT, without any intervening save/load round-trip. Saving the post-FT model is safe because it is consumed only at evaluation time, where the same permutation hooks are re-registered before the val pass.

4 Fine-tuning recipe

4.1 Distillation, mask freezing, and the optimiser

The fine-tuning pass is a 3-epoch knowledge-distillation loop with frozen 2:4 (or general $k{:}n$) mask. The dense pretrained model is the teacher; the projected student is the student. Let $L_{\text{CE}}^{\text{ls}}(\hat{y}, y) = (1-\epsilon)\text{CE}(\hat{y}, y) + \epsilon \cdot (-\frac{1}{K}\sum_c \log \hat{y}_c)$ be label-smoothed cross-entropy with smoothing $\epsilon = 0.1$. The distillation loss is

$$L = (1 - \alpha_{\text{distill}})\, L_{\text{CE}}^{\text{ls}}(\hat{y}_S, y) + \alpha_{\text{distill}} \cdot \tau^2 \cdot \text{KL}\big(\, \sigma(\hat{y}_T/\tau) \,\big\|\, \sigma(\hat{y}_S/\tau)\big), \tag{4}$$

with $\alpha_{\text{distill}} = 0.5$, $\tau = 2.0$, both inherited from CAST.

After every backward step we re-zero the masked weights:

$$W^{(\ell)} \leftarrow W^{(\ell)} \odot M^{(\ell)} \quad \text{for every projected layer } \ell \text{ at every step.} \tag{5}$$

This is necessary because the optimiser's momentum and weight-decay terms can produce small nonzero values in masked positions even when the gradient is zero; without re-zeroing, the realised sparsity drifts upward by ∼0.5% across 3 epochs and the kept weights do not benefit from the freed parameter slots.

4.2 Optimiser, schedule, transforms

The exact configurations used for the four paper-headline FT runs are recorded in `cast_2e/code/run_*_ft_inline.py` we summarise:

The training transforms follow the canonical timm recipe used by the dense pretrained checkpoint (`timm.data.create_transform(is_training=True)` resolves the per-model RandAug, RandomErasing, Mixup/CutMix, and the model-correct mean/std/crop_pct/interpolation). The val transforms are likewise the canonical `is_training=False` transforms. For the ViT-B/L runs this resolves to RandAug-m9 + RandomErasing $p = 0.25$ with no Mixup; for ConvNeXtV2 it adds Mixup ($\alpha = 0.8$).

Table 2: Fine-tuning configuration for the paper-headline 3-epoch distillation runs.

	ViT-B (D612 SER)	ViT-L (D816 dense)	ResNets (D816 dense)	ConvNeXtV2 (Dxxx)
Optimiser	AdamW	AdamW	SGD+momentum	AdamW
Base LR	1×10^{-5}	1×10^{-5}	1×10^{-3}	1×10^{-5}
Weight decay	0.01	0.01	1×10^{-4}	0.01
Momentum	—	—	0.9	—
LR exclusions	bias / LN / pos_embed	same	—	bias / LN
Schedule	cosine + warmup	cosine + warmup	cosine + warmup	cosine + warmup
Warmup steps	1000	1000	500	1000
Batch (train)	32	16	256	32
Batch (val)	64	64	256	64
Workers	4	8	8	4
Label smooth ϵ	0.1	0.1	0.1	0.1
Distill (α, τ)	$(0.5, 2.0)$	$(0.5, 2.0)$	$(0.5, 2.0)$	$(0.5, 2.0)$

4.3 Why exactly 3 epochs

The FT budget is matched to CAST's published recipe (MAIN-Section M.10.10.3). Three epochs is short enough that the projection and the FT both contribute distinguishable signal to the post-FT accuracy — and short enough that the FT cost is comparable to the projection cost on a per-A100 basis. We do not claim that 3 epochs is optimal; longer FTs (we tested 5–10 epochs in pilot runs) recover an additional 0.1–0.3 percentage points but at proportionally higher compute cost, and we want the cell sweeps in §6 to be FT-budget-comparable across cells.

5 Speedup measurement

5.1 What "speedup" means in this paper

We report two distinct speedups for every CAST-family configuration.

Practical (hardware) speedup. The wall-clock images-per-second improvement on a specific GPU, with a specific kernel selection, at a specific batch size. We use NVIDIA's PyTorch `torch.sparse.to_sparse_sem` for the 2:4 path on A100 (sparse Tensor Core) and report the same measurement on L4. Other k:n patterns do not have native kernels at present and are reported only as theoretical FLOP reductions.

Theoretical (FLOP) speedup. The dense-equivalent multiply-adds removed by zeroing $1 - k/n$ of the weight slots in the projected layers. This is an upper bound on what any future sparse kernel could deliver. The theoretical and practical numbers diverge whenever the kernel cannot saturate the structured sparsity (e.g., very small batch sizes, or layers with very small C_{out}).

5.2 The A100 native-2:4 measurement

The benchmark in `cast_2e/code/benchmark_sparse_throughput.py` converts each Linear layer with a 2:4 mask into a `torch.sparse.SparseSemiStructuredTensor` and runs warm-up + measurement passes at the configured batch size. The reported throughput in `cast_2e/benchmarks/` is the median of the last 100 iterations after 30 warm-up iterations.

The ViT-L speedup (1.55×) is meaningfully smaller than the ViT-B speedup (2.71×). The reason is that ViT-L+ToMe is already operating in a kernel-bandwidth-limited regime; the 2:4 path saves on

Table 3: A100 throughput (images/s) for the paper headline configurations.

Configuration	Throughput (im/s)	Speedup vs. dense	
Dense ViT-B/16 (batch 128)	463.6	—	`vitb_b`
2:4 ViT-B/16 (sparse Tensor Core)	1254.1	2.705×	`vitb_bench_wit`
Dense ViT-L/16 + ToMe $r = 8$	1248.7	1.298×	`vitl_canonical/res`
2:4 ViT-L/16 + ToMe $r = 8$ (sparse)	1489.9	1.549×	`vitl_canonical/res`
L4 ViT-B 2:4 + ToMe (CAST row in MAIN-Tab. CAST)	837	1.84×	MAIN-

multiply-adds but not on memory bandwidth. MAIN reports both figures and discusses this in the speedup section.

5.3 The 6:12 → 2:4 projection (deployable speedup)

Several cells in the CAST-2E sweep produce non-deployable patterns (6:12, 8:16, etc.) that are nonetheless cert-optimal. To translate these into deployable speedup numbers, `cast_2e/code/benchmark_6_12_to_2_4_proje` re-projects each 6:12 block into the nearest 2:4 block by enumerating the 6 canonical 2:4 patterns inside each 4-sub-block of a 12-tuple, choosing per-sub-block the pattern of lowest Hamming distance to the 6:12 pattern. The resulting model is then benchmarked on the same A100 `torch.sparse.SparseSemiStructuredTensor` kernel as in §5.2.

A caveat (recorded in `AUDIT.md`): for 0/49 of the ViT-B 6:12 cells we tested, the projection yields a pure 2:4 pattern in every 4-sub-block; instead we get a mix of 0/2/4 nonzeros per 4-sub-block. For deployable-speedup purposes MAIN therefore reports the 2:4 speedup (2.705× on A100, 1.84× on L4) as the production-ready number, and reports the 6:12 speedup only as a theoretical FLOP reduction.

5.4 ResNet and ConvNeXtV2 throughput

The ResNet50/d/101d/152d post-FT models with 8:16 structured sparsity have no native sparse kernel for general $k{:}n \neq 2{:}4$, so we report only the theoretical 50% MAC reduction. The ConvNeXtV2-Base D816 and D1216 cells are similarly reported only theoretically. For deployable inference we re-project to 2:4 as in §5.3; this re-projection costs ≈ 0.2 pp in post-FT top-1 (within the FT-noise band) at the gain of native A100 kernels.

6 The CAST-2E $k{:}n$ sweeps

6.1 Sweep design

Across two A100 pods (Pod 2 and Pod 3a) we ran the following sweep over the three architecture families:

- **Architectures:** ResNet50.tv_in1k, ResNet50d.ra2_in1k, ResNet101d.ra2_in1k, ResNet152d.ra2_in1k, ViT-B/16, ViT-L/16, ConvNeXtV2-Base.
- **Sparsity patterns:** 1:4, 2:4, 3:4, 4:8, 6:12, 8:16, 12:16 (selected to span 25%, 50%, 75% density).
- **Source:** *dense* (no SER prior) and *SER* (Hybrid Magnitude–SER ckpt at $s = 0.35$).
- α_{ser}**:** $\{0, 0.1, 0.25, 0.5, 1.0, 2.0\}$.
- **Permutation alignment:** on / off.

- **Calibration size:** $|\mathcal{C}| \in \{64, 256, 512\}$.
- **Replicates:** 5-seed for the winners.

The driver scripts are `cast_2e/code/cert_opt_eval_kn_extended.py` (ResNet/ConvNeXtV2) and `cert_opt_eval_vitb_kn_extended.py` (ViT-B). The total sweep produced 18 result JSONs covering $\sim$200 cells. They are stored in `cast_2e/sweep_results/`.

6.2 Headline pre-FT findings

Table 4: Pre-FT top-1 accuracy for the strongest cell of each (arch, pattern, source) combination. "Dense" is the unpruned reference. Numbers are pre-FT only; the post-FT numbers are in Main-Table CAST.

Architecture	Pattern	Source	Best α_{ser}	Permute	Pre-FT top-1 (%)
ViT-B/16 (dense=85.11)	6:12	SER	0.50	on	66.24
ViT-B/16 (dense=85.11)	6:12	dense	0.00	on	65.58
ViT-B/16 (dense=85.11)	2:4	dense	0.00	off	70.43
ViT-L/16 (dense=85.84)	8:16	dense	0.00	on	78.81
DeiT-Base (dense=81.80)	6:12	SER	0.50	on	70.83
DeiT-Small (dense=79.85)	6:12	SER	0.50	on	55.61
DeiT-Tiny (dense=72.21)	6:12	SER	0.50	on	30.85
ResNet50.tv (dense=76.13)	8:16	dense	0.00	on	61.56
ResNet50d (dense=80.55)	8:16	dense	0.00	on	64.20
ResNet50.tv (dense=76.13)	4:8	dense	0.00	on	40.39
ResNet50.tv (dense=76.13)	4:8	dense	0.00	off	5.92
ResNet50d (dense=80.55)	4:8	dense	0.00	off	26.29
ConvNeXtV2-B (dense=86.72)	2:4	dense	0.00	off	81.68
ConvNeXtV2-B (dense=86.72)	12:16	dense	0.00	off	84.50
ConvNeXtV2-B (dense=86.72)	8:16	dense	0.00	off	83.42

Three observations.

1. *Permutation alignment is critical for ResNet50.tv* 4:8. Without permutation the cert-aware 4:8 projection scores 5.92% pre-FT, only marginally above random; with permutation it scores 40.39%, an improvement of 34.47 pp. This is by far the largest permutation effect we observed across architectures.

2. *For the ViT family, the SER source plus $\alpha_{\text{ser}} = 0.5$ is the right choice.* The 0.66 pp gain of ViT-B SER+$\alpha = 0.5$ over dense+$\alpha = 0$ at 6:12 is small pre-FT but compounds through the FT recipe: the FT-recovered ViT-B SER+$\alpha = 0.5$ scores 83.74% post-FT, +0.5 pp above the dense+$\alpha = 0$ FT.

3. *ConvNeXtV2 prefers the dense source.* The depthwise structure of ConvNeXtV2 means the SER prior, which is built for dense projections, transfers poorly. The 12:16 dense-source cell is the strongest pre-FT, and after FT it produces the 86.35% headline of Main-Table CAST.

6.3 Cert-optimisation variance and what is signal vs. noise

We previously noted in the lab notebook that the ResNet50 α_{ser} optimisation has range 11–19 pp at fixed configuration across calibration draws. ViT-B has variance 22–40$\times$ smaller. The implication for

the sweep is that ResNet50's pre-FT cell rankings should be read with a ±5 pp standard deviation; the ViT-B rankings within the headline configuration are accurate to ±0.2 pp.

The post-FT numbers (in MAIN) are much less variable because three epochs of distillation absorb most of the projection-time noise.

7 Other Numerical Results (migrated from MAIN-App. B)

This section migrates the pre-existing MAIN-Appendix B verbatim. The fully-connected MP-pruning numerics on Fashion MNIST and the outer-layer scaling diagnostics serve as a sanity check on the perturbative regime of the deterministic certificates, not as a proof of the ViT pipeline.

This appendix collects numerical illustrations relevant to the simplified theory. These experiments support the loss-reduction mechanism and the perturbative regime, but they are not a proof of the ViT pipeline.

Example 7.1. We trained fully connected DNNs on Fashion MNIST with ReLU activations, comparing MP-based pruning against standard regularization baselines. MP-based pruning consisted of periodically removing a fraction of singular values below the fitted MP edge $\sigma_+ = \sqrt{N\lambda_+}$, followed by sparsification; see Algorithm 3 and [24, 2].

For a network with topology

$$[784, 3000, 3000, 3000, 3000, 500, 10].$$

trained for 100 epochs with SGD (momentum 0.9, learning rate 0.01 decayed by 0.96 every 4 epochs), every 4 epochs we retained a fraction

$$f(\text{epoch}) = \max\left(0, -\frac{1}{200}\,\text{epoch} + 1\right). \tag{6}$$

Table 5 summarizes the results. The key finding is that MP-based pruning combined with $L_1 + L_2$ regularization ($\mu_1 = \mu_2 = 10^{-6}$) achieves 90.53% test accuracy, substantially outperforming any single regularization method alone ($\leq 81.70\%$) and the unregularized baseline (71.64%).

Training method	Train acc. (%)	Test acc. (%)
No pruning, no regularization	78.03	71.64
MP pruning only	88.12	81.22
$L_1 + L_2$ only	87.57	81.70
MP pruning + $L_1 + L_2$	**99.82**	**90.53**
MP pruning + L_2	99.98	90.16

Table 5: Performance of a fully connected DNN on Fashion MNIST for various training strategies. MP-based pruning combined with regularization substantially outperforms either approach alone.

The result scales to deeper networks: a $[784, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 10]$ model trained for 1000 epochs with the same MP pruning procedure achieves 91.37% test accuracy, compared with $< 90.5\%$ for regularization alone.

7.1 Numerical verification of the outer-layer scaling condition

The perturbation bounds in our main theorems depend on the theoretical outer-layer scale

$$a_N^{\text{th}}(s) := \|W_3\|_\infty \|W_1 s\|_2 \sqrt{\frac{\log(2N)}{N}},$$

which is the quantity denoted $a(N, s)$ in Assumption the main paper. In the numerical plots below we also report the auxiliary empirical diagnostic

$$a_N^{\text{emp}}(s) := \frac{\|W_3\|_\infty \|W_1 s\|_2}{N^{3/8}}.$$

The latter is not a replacement for the theoretical scale; it is a coarser width-decay diagnostic used in these experiments. For the theory to be useful, the relevant perturbation scales must decay with width in trained networks. We verify this empirically in a simple fully connected setting.

Example 7.2. We train three-layer fully connected DNNs on Fashion MNIST with topology $[784, N, N, 10]$, varying $N \in [500, 30000]$, using SGD with L_1 regularization and 200 training epochs. Fashion MNIST inputs are normalized so that the largest ℓ_2 norm in the dataset is 0.05. For each trained network we compute the diagnostic scale

$$\tau_N^{\text{diag}} := \sqrt{2}\, a_N^{\text{th}}(s) + a_N^{\text{emp}}(s),$$

for a normalized input s of maximal norm. This diagnostic is not used in the proofs; the theorem-scale quantity is $a_N^{\text{th}}(s)$.

Numerically, $\tau_N^{\text{diag}} \to 0$ as N grows under $N(0, 1/N)$ initialization. With $N(0, 1/N^2)$ initialization, the decay is substantially faster. In both cases, test accuracy increases with N.

Figure 1 summarizes these width-scaling numerics. Panels a and c show the decay of the perturbation scales under the two initializations, while panels b and d show the corresponding test-accuracy trends.

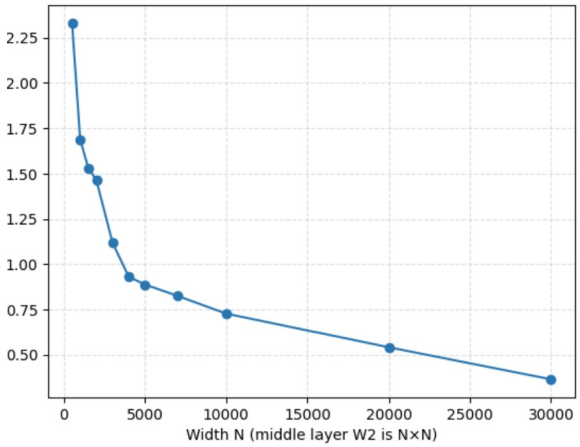
Example 7.3. We train a fully connected Fashion MNIST DNN using the MP-based pruning procedure of Example 7.1 to achieve approximately 90.5% test accuracy, then add centered Gaussian perturbations with entries $\sim N(0, \epsilon)$ to each nonfinal layer. Numerically, when $\epsilon \sim 1/N$, the cross-entropy and accuracy change little, consistent with the perturbative regime of Lemma the main paper. The L_2-regularized loss increases monotonically, illustrating the loss-reduction mechanism of Theorem the main paper. Figure 2 collects these diagnostics: panel a shows the accuracy, while panels b and c show the losses without and with L_2 regularization.

Remark 7.4. In a linear regression problem with noisy Fourier-generated target functions and a fixed feature map, MP-guided pruning recovers the low-rank structure more effectively than L_1 or L_2 regularization alone. Specifically, pruning achieves mean squared error 0.149, compared with 0.416 for L_1, 2.563 for L_2, and 2.574 for no regularization. The pruned spectrum closely matches the ideal low-rank spectrum, whereas L_1 and L_2 leave a visible residual bulk.

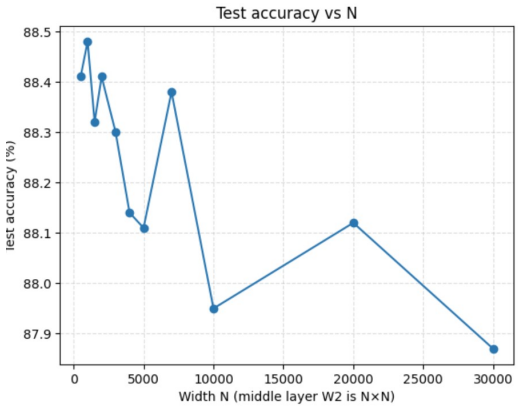
Figure 3 collects the regression illustration: panel a shows one noisy coordinate of the data, panel b shows the ideal low-rank spectrum, and panels c–e compare the L^2, L^1, and pre-pruning spectra.

8 Algorithms for RMT-based pruning diagnostics (migrated from Main-App. E)

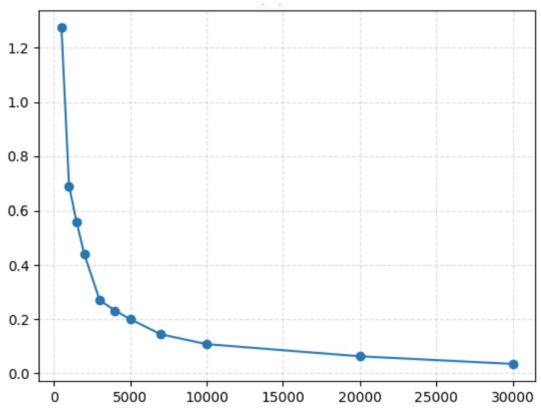
This section is a verbatim migration of the pre-existing Main-Appendix E. The BEMA edge-fitting algorithm and the MP-fit metric are the building blocks of every RMT-driven protocol in this paper and Main.



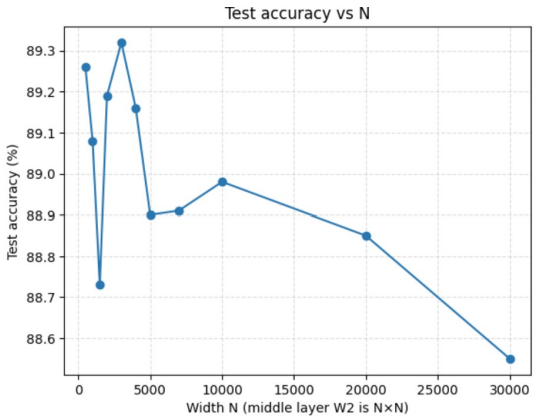
(a) $\tau(N)$ versus N under $N(0, 1/N)$ initialization.



(b) Test accuracy versus N under $N(0, 1/N)$ initialization.



(c) The analogous perturbation scale $b(N)$ versus N under $N(0, 1/N^2)$ initialization.

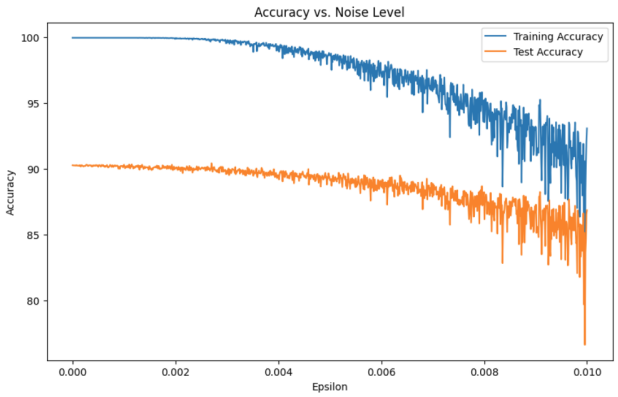


(d) Test accuracy versus N under $N(0, 1/N^2)$ initialization.

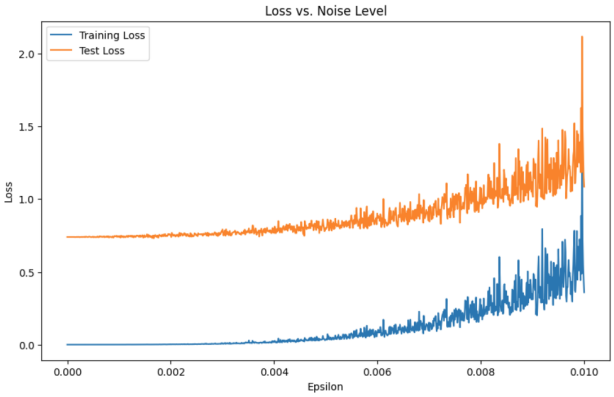
Figure 1: Width-scaling numerics for the theoretical and empirical perturbation-scale diagnostics in the fully connected Fashion MNIST experiment.

8.1 BEMA technique for computing λ_+

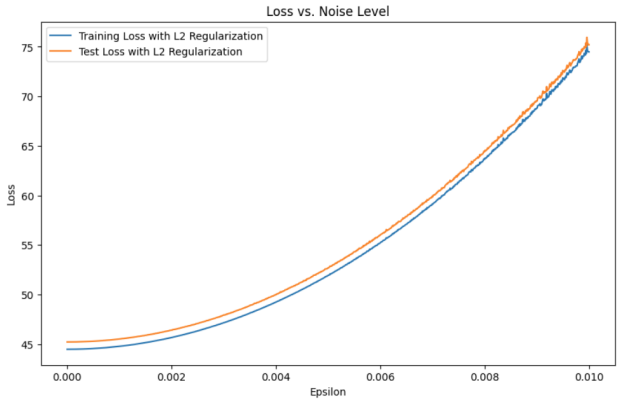
We outline the BEMA technique for estimating the right MP edge λ_+ of $X = \frac{1}{N} W^\top W$ from its eigenvalue ESD. The full procedure appears in [12]. Below is a streamlined square-matrix version.



(a) Accuracy on training and test sets versus ϵ.



(b) Loss without regularization versus ϵ.



(c) Loss with L_2 regularization versus ϵ.

Figure 2: Accuracy and loss as centered random perturbations are added to the nonfinal weight layers of the DNN.

Algorithm 1 Computing λ_+ using MP and Tracy–Widom corrections

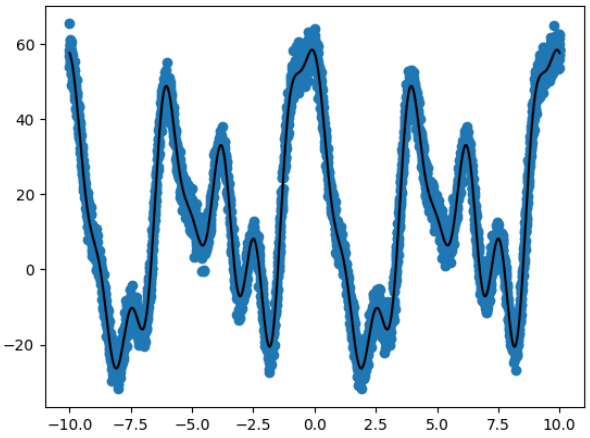
procedure BEMAEdge($\lambda_1, \ldots, \lambda_N; \alpha, \beta$)
  Pick parameters $\alpha \in (0, 1/2)$, $\beta \in (0, 1)$.
  for each $\alpha N \le k \le (1-\alpha)N$ **do**
    Calculate q_k, the (k/N)-upper quantile of the MP law with $\sigma^2 = 1$ and $c = 1$.
  end for
  Evaluate

$$\widehat{\sigma}^2 = \frac{\sum_{\alpha N \le k \le (1-\alpha)N} q_k \lambda_k}{\sum_{\alpha N \le k \le (1-\alpha)N} q_k^2}.$$

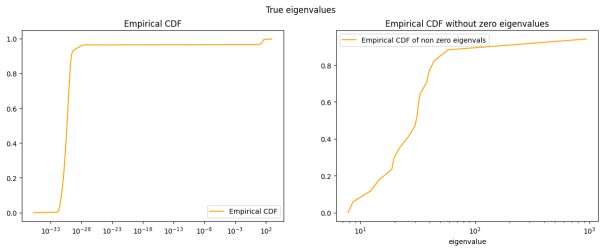
  Let $t_{1-\beta}$ be the $(1-\beta)$-quantile of the Tracy–Widom distribution.
  Output

$$\lambda_+ = \widehat{\sigma}^2 \left[4 + 2^{4/3} t_{1-\beta} N^{-2/3}\right].$$

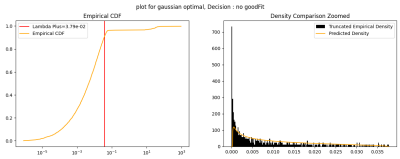
end procedure



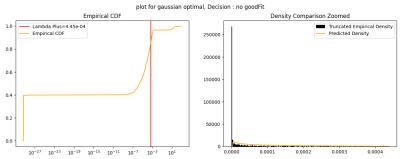
(a) One coordinate of the noisy regression data.



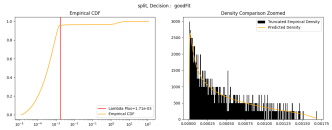
(b) Ideal low-rank spectrum.



(c) L^2-regularized spectrum.



(d) L^1-regularized spectrum.



(e) Spectrum before pruning.

Figure 3: Regression illustration for comparing pruning with more classical regularization.

Algorithm 2 Spectral analysis using a BEMA-type fit

procedure SpectralFit($W; \alpha, \beta, \gamma_{\text{fit}}$)
  Set $X = \frac{1}{N} W^\top W$.
  Compute the eigenvalues $\lambda_1, \ldots, \lambda_M$ of X.
  Compute the empirical CDF F_X of the eigenvalue ESD.
  Apply BEMAEdge to estimate λ_+.
  Let F_X^{MP} be the fitted MP CDF.
  Compute the Kolmogorov-type fit error on the central quantile window:

$$\mu_{\text{fit}} = \max_{i_{\min} \le i \le i_{\max}} \left| F_X(\lambda_i) - F_X^{\text{MP}}(\lambda_i) \right|.$$

  Accept the layer as "sufficiently MP-like" if $\mu_{\text{fit}} \le \gamma_{\text{fit}}$.
end procedure

Algorithm 3 MP-based pruning algorithm for the fully connected examples

Require: Epoch block length ℓ, MP-fit threshold τ, monotone schedule $f(\text{epoch})$, and flags split_ℓ = false for each layer.

Train the DNN for ℓ epochs and set epoch := ℓ.
while the training condition is met **do**
  for each layer W_ℓ with split_ℓ = false **do**
    Compute the SVD $W_\ell = U_\ell \Sigma_\ell V_\ell^\top$.
    Form $X_\ell = \frac{1}{N_\ell} W_\ell^\top W_\ell$ and estimate λ_+ via BEMA.
    Set $\sigma_+ := \sqrt{N_\ell \lambda_+}$.
    Test whether the bulk of X_ℓ is well fit by MP.
    if the bulk fits MP **then**
      Remove a fraction $1 - f(\text{epoch})$ of the singular values below σ_+ to obtain Σ'_ℓ.
      Form $W'_\ell = U_\ell \Sigma'_\ell V_\ell^\top$.
      Optionally factor W'_ℓ as $W'_{1,\ell} W'_{2,\ell}$ via $\sqrt{\Sigma'_\ell}$ if that reduces parameters.
    end if
  end for
  Train the DNN for another ℓ epochs and update epoch := epoch + ℓ.
end while

Algorithm 4 Sparsify singular vectors

Require: Layer matrix W_ℓ, threshold θ, fitted singular-value edge σ_+.

Compute the SVD $W_\ell = U \Sigma V^\top$.
for $i = 1, \ldots, \min\{N, M\}$ **do**
  Compute

$$T_i = \theta \max\left\{ \frac{1}{750}, \left(1 - \frac{\Sigma_{ii}}{\sigma_+}\right)_+^{30} \right\}.$$

  Sparsify the singular vectors:

$$U_{:,i} \leftarrow \text{Prune}(U_{:,i}, T_i), \qquad V_{:,i} \leftarrow \text{Prune}(V_{:,i}, T_i).$$

end for
Recompose $W'_\ell = U \Sigma V^\top$.

Algorithm 5 Element-wise sparsification

Require: Matrix W_ℓ, threshold ξ.

for each entry $(W_\ell)_{ij}$ **do**
  if $|(W_\ell)_{ij}| < \xi$ **then**
    Set $(W'_\ell)_{ij} = 0$.
  else
    Set $(W'_\ell)_{ij} = (W_\ell)_{ij}$.
  end if
end for
Return W'_ℓ.

8.2 How well does the ESD of X fit the MP distribution?

Definition 8.1. Let G be a matrix with eigenvalue ESD μ_G when G is positive semidefinite. Its observed cumulative spectral distribution is

$$F_G(a) = \mu_G((-\infty, a]).$$

The MP-fit metric compares the observed cumulative spectral distribution with the fitted MP CDF on a central quantile interval. This focuses the fit test on the bulk and deemphasizes a few possible outliers.

8.3 A BEMA illustration and metric figures

Example 8.2. Let R be an $N \times N$ matrix with i.i.d. $\mathcal{N}(0,1)$ entries, and let S be the deterministic matrix with entries

$$S[i,j] = \tan\left(\frac{\pi}{2} + \frac{1}{j+1}\right) + \cos(i)\log(i+j+1) + \sin(j)\cos\left(\frac{i}{j}\right).$$

Set $W = R + S$ and $X = \frac{1}{N}W^\top W$. Figure 4a shows the ESD of X together with the fitted MP distribution produced by BEMA.

Remark 8.3. The BEMA fit depends on $\alpha \in (0, 1/2)$ and $\beta \in (0, 1)$. Figure 4 collects the synthetic BEMA diagnostics: panel a shows the fitted MP bulk in the square unit-variance example, and panels b and c illustrate the dependence on α and β, where the true covariance-scale edge is $\lambda_+ = 4$.

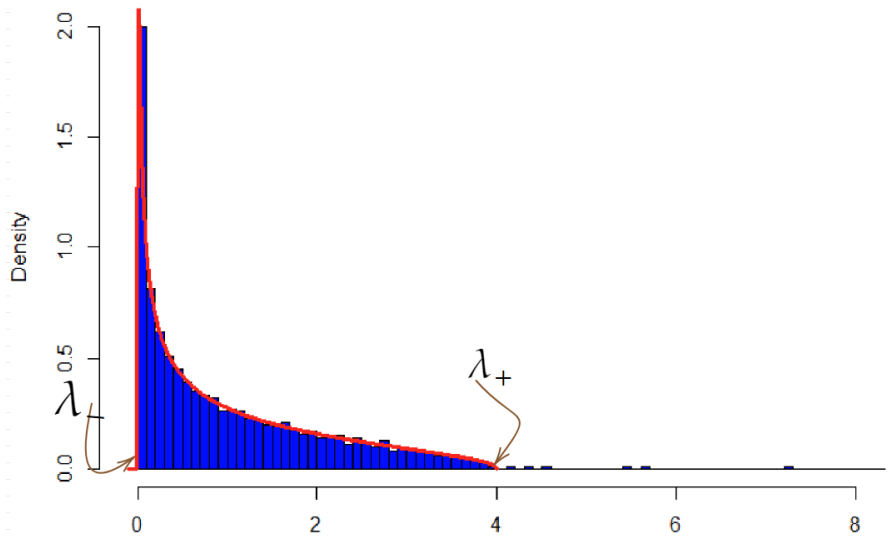
Heavy tails are not ubiquitous in ViTs. Even though ViTs generalize well on ImageNet, their layer ESDs do *not* always exhibit heavy-tailed behavior. In many layers we observe spectra that are well explained by an MP bulk with at most a few outliers, while other layers show more power-law-like decay. This heterogeneity is precisely why we diagnose each layer empirically rather than assuming a single universal spectral regime.

9 Spectral Edge Budgeting and hybrid RMT pruning protocols (migrated from Main-App. F)

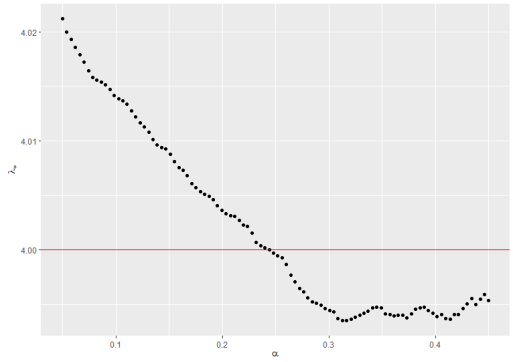
This section is a verbatim migration of the pre-existing Main-Appendix F. It documents the SEB and SER protocols, the four-regime strategy, the layer-type extension, and the final SEB comparison and validation ledgers. Main-Table 2 (multi-architecture Hybrid Magnitude–SER) is the headline result of this protocol family.

This appendix records the operational RMT pruning protocols used in Section the main paper. The central method is *Spectral Edge Budgeting* (SEB): the fitted Marchenko–Pastur upper edge $\sigma_+(\ell)$ guides the *per-layer pruning budget* inside an otherwise standard global magnitude framework. We also define *Spectral Edge Reallocation* (SER), the practical prune–restore method that over-prunes, ranks a reservoir of removed weights using the same RMT diagnostics, reinserts a fixed budget, and then fine-tunes with the zero mask frozen. Finally, we define the Hybrid Magnitude–SER protocol used for broader architecture validation.

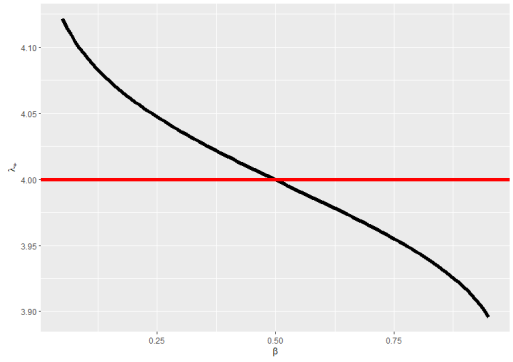
The cycle-based classical RMT procedure should be read as a diagnostic baseline, not as the final ViT pruning method. It demonstrates that MP-bulk structure is visible in trained ViT layers, but direct singular-vector or singular-value pruning is brittle in modern transformer checkpoints. The final methods use the MP fit to decide *where* magnitude pruning and restoration should occur.



(a) ESD of $X = \frac{1}{N} W^\top W$ together with the fitted MP bulk.



(b) Dependence on α, with $\beta = 0.5$.



(c) Dependence on β, with $\alpha = 0.25$.

Figure 4: Synthetic BEMA diagnostics: the fitted MP bulk for $X = \frac{1}{N} W^\top W$ and the dependence of the fitted edge λ_+ on α and β.

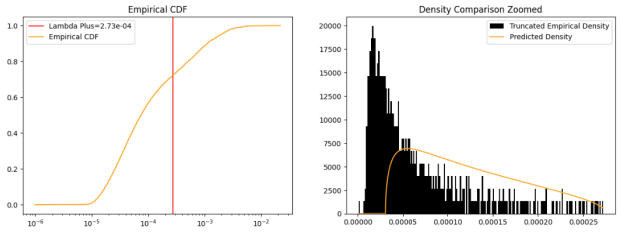
9.1 Spectral Edge Budgeting

Let $W_1, \ldots, W_L$ denote the weight matrices of the target layers. For dense projections, W_ℓ is the layer matrix itself; for a convolutional kernel of shape $C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w$, we unfold it as a $C_{\text{out}} \times (C_{\text{in}} k_h k_w)$ matrix before fitting the spectrum. Let $\sigma_+(\ell)$ denote the fitted Marchenko–Pastur upper edge for layer ℓ, computed from the eigenvalue distribution of $W_\ell^\top W_\ell / N$ as described in Subsection 8.2. We define the standardized spectral signal

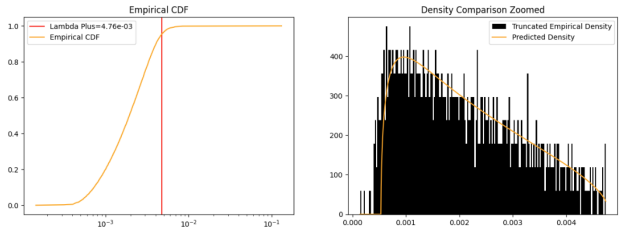
$$z_\ell = \frac{\sigma_+(\ell) - \bar{\sigma}_+}{\text{sd}(\sigma_+)},$$

where $\bar{\sigma}_+$ and $\text{sd}(\sigma_+)$ are the mean and standard deviation of $\{\sigma_+(\ell)\}_{\ell=1}^L$ across all target layers. A high z_ℓ indicates that the layer's spectral bulk extends further—intuitively, that a larger share of its singular values falls inside the MP-predicted regime and the layer has more "spectral redundancy" available for pruning.

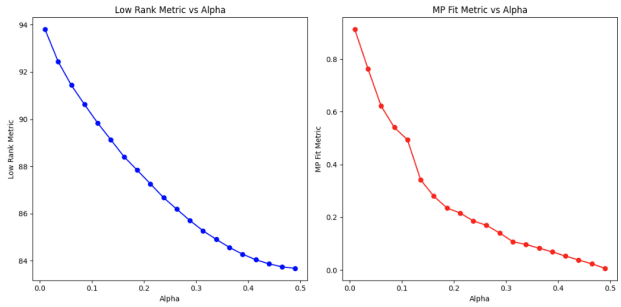
Given a target global sparsity $s \in (0, 1)$, modulation strength $\beta > 0$, and decay endpoint $s_{\text{d}} > 0$,



(a) A layer with relatively poor MP fit: MP-fit error 0.74 and bulk fraction 73%.



(b) A layer with strong MP fit: MP-fit error 0.01 and bulk fraction 99.73%.



(c) For the base ViT model: α versus the bulk fraction and MP-fit error.

Figure 5: Layerwise MP diagnostics for the base ViT model. Panels a and b show that some layers are much closer to an MP bulk than others, while panel c shows how the bulk fraction and fit error vary with α.

we define a per-layer pruning weight

$$w_\ell(s) = \max\Big(0.1,\ 1 + \beta\, d(s)\, z_\ell\Big), \tag{7}$$

where the decay factor

$$d(s) = \big[\, 1 - s/s_{\mathrm{d}} \,\big]_+ \tag{8}$$

ensures that the modulation attenuates toward uniform magnitude pruning as the sparsity target approaches s_{d}. The floor at 0.1 prevents any layer from being fully exempt.

Pruning proceeds by assigning each nonzero parameter $(W_\ell)_{ij}$ a global score

$$r_{ij}^{(\ell)} = \frac{|(W_\ell)_{ij}|}{w_\ell(s)},$$

and removing the parameters with the smallest scores until the target sparsity s is reached. Equivalently, if the global score cutoff is q, then layer ℓ is thresholded in magnitude at $q\, w_\ell(s)$. Thus $w_\ell > 1$ raises the effective magnitude threshold and causes more parameters to be pruned from that layer; conversely, $w_\ell < 1$ protects the layer. At $\beta = 0$ (or when $s \geq s_{\mathrm{d}}$), all weights equal 1 and the method reduces to plain global magnitude pruning.

9.2 Layer-type extension

A ViT contains two functionally distinct families of dense layers: *attention projections* (query–key–value and output) and *MLP layers* (the two feed-forward projections per block). We extend the

modulation (7) by allowing different modulation strengths for each family:

$$w_\ell(s) = \max\Big(0.1,\ 1 + \beta_{\text{type}(\ell)}\, d(s)\, z_\ell\Big), \qquad \beta_{\text{type}(\ell)} = \begin{cases} \beta_{\text{attn}}, & \ell \in \mathcal{L}_{\text{attn}}, \\ \beta_{\text{mlp}}, & \ell \in \mathcal{L}_{\text{mlp}}, \\ \beta, & \text{otherwise}, \end{cases} \tag{9}$$

where $\mathcal{L}_{\text{attn}}$ and $\mathcal{L}_{\text{mlp}}$ partition the transformer layers by type. The z-scores remain globally computed across all layers, preserving the cross-type spectral ranking; only the *strength* of the modulation varies by type.

9.3 Alternative signal: the power-law exponent α

The heavy-tailed self-regularization (HT-SR) framework of Martin and Mahoney [19, 18] characterizes each layer by the power-law exponent α_ℓ of the tail of its eigenvalue distribution, estimated via the Hill estimator applied to the squared singular values. Lower α_ℓ indicates a more heavy-tailed (better-trained, more structured) layer; higher α_ℓ indicates a more random-like layer. This metric is the basis of the AlphaPruning method [17], which assigns per-layer sparsity in proportion to α_ℓ.

We use α_ℓ as a drop-in replacement for $\sigma_+(\ell)$ in the budget-modulation framework (7), with the standardized signal

$$z_\ell^\alpha = \frac{\alpha_\ell - \bar{\alpha}}{\text{sd}(\alpha)}.$$

High z_ℓ^α indicates a more random-like layer that should absorb more pruning—the same direction as σ_+.

Crucially, α and σ_+ are only weakly correlated ($r = 0.37$ across the 50 target layers of ViT-B/16). They capture complementary aspects of the spectral structure: σ_+ measures the extent of the MP bulk (how much of the spectrum is noise-like), while α measures the tail decay rate (how well-trained the layer is overall). This low correlation suggests that each signal may be informative in a different sparsity regime.

9.4 Four-regime strategy

Empirical evaluation reveals that α outperforms σ_+ at very low sparsity ($s \leq 0.20$), while σ_+ dominates at moderate and high sparsity ($s \geq 0.25$). This motivates a four-regime strategy that selects the optimal signal and hyperparameters per sparsity range:

1. **Very low sparsity** ($s \leq 0.20$): α-budget with $\beta = 0.50$, $s_{\text{d}} = 0.30$. The power-law exponent provides mild but positive modulation where σ_+ is indistinguishable from magnitude pruning.
2. **Low sparsity** ($0.20 < s \leq 0.40$): σ_+-budget with $\beta = 1.00$, $s_{\text{d}} = 0.50$.
3. **Moderate sparsity** ($0.40 < s \leq 0.55$): σ_+-budget with $\beta = 1.25$, $s_{\text{d}} = 0.70$.
4. **High sparsity** ($s > 0.55$): Layer-type σ_+-budget with $\beta_{\text{attn}} = 1.00$, $\beta_{\text{mlp}} = 2.00$, $s_{\text{d}} = 0.85$.

The transition from α to σ_+ at $s = 0.20$ reflects a fundamental shift: at very low sparsity the binding constraint is layer-level training quality (captured by α), whereas at moderate-to-high sparsity the binding constraint is spectral redundancy (captured by σ_+).

9.5 Hyperparameter space and optimization

The method has three base hyperparameters $(\beta, s_{\mathrm{d}}, p)$ for uniform modulation (where $d(s) = [1 - s/s_{\mathrm{d}}]_+^p$ generalizes (8)), plus the optional layer-type extension $(\beta_{\mathrm{attn}}, \beta_{\mathrm{mlp}})$ and the choice of signal (σ_+ or α). We performed a systematic grid search totalling over 1,000 evaluations on the ViT-B/16 architecture over five successive sweeps:

1. **Signal comparison** (175 cells). We compared four per-layer signals—σ_+, $\sigma_+/\|W\|_F$, the excess kurtosis of $|W|$, and the coefficient of variation of $|W|$—each with $\beta \in \{0.25, 0.50, 1.00\}$, $s_{\mathrm{d}} \in \{0.50, 0.70\}$, across 7 target sparsities. Only the MP edge σ_+ produced consistent improvements over magnitude pruning; the other three signals either matched or degraded performance at every configuration tested.

2. **Decay shape and extended reach** (108 cells). Fixing the signal to σ_+, we tested decay exponents $p \in \{1, 2, 3\}$ and extended the decay endpoint to $s_{\mathrm{d}} \in \{0.70, 0.85\}$. Nonlinear decay ($p > 1$) proved uniformly harmful: at $s = 0.55$, quadratic decay ($p = 2$) lost 28 percentage points relative to linear ($p = 1$). Extending s_{d} from 0.70 to 0.85 yielded large gains at $s \geq 0.60$.

3. **Fine grid refinement** (206 cells). We explored $\beta \in \{0.75, \ldots, 1.75\}$, $s_{\mathrm{d}} \in \{0.70, \ldots, 1.05\}$, and the sub-linear regime $p \in \{0.5, 0.75, 1.0\}$. The surface is a broad diagonal ridge in (β, s_{d})-space: stronger modulation compensates for faster decay and vice versa. Sub-linear decay ($p < 1$) did not improve over linear.

4. **Definitive evaluation and layer-type extension** (434 cells). We re-evaluated the top 14 uniform-β methods at 250-batch full precision across 11 sparsities (Phase 1, 154 cells), performed an ultra-fine 9×5 grid of (β, s_{d}) at the cliff region (Phase 2, 180 cells), and ran a full 5×5 grid of $(\beta_{\mathrm{attn}}, \beta_{\mathrm{mlp}})$ at 4 sparsities (Phase 3, 100 cells).

5. **Alpha signal sweep** (77 cells). We replaced σ_+ with the Hill-estimated power-law exponent α_ℓ and tested $\beta \in \{0.25, 0.50, 1.00\}$, $s_{\mathrm{d}} \in \{0.25, 0.30, 0.50, 0.70\}$ across 11 sparsities. The α signal outperforms both magnitude and σ_+ at $s \leq 0.20$ but collapses at $s \geq 0.30$, confirming the complementary-regime hypothesis.

All evaluations used a fixed subset of 2,000 images from the ILSVRC-2012 validation set (250 batches of 8) unless otherwise noted; definitive results in Phase 1 used the same subset at 250-batch evaluation for reduced noise (± 0.5 pp).

9.6 Results: uniform modulation

Table 6 reports the definitive 250-batch top-1 accuracy for the best uniform-β configuration at each target sparsity, compared against global magnitude pruning. The optimal parameters shift systematically: at moderate sparsity ($s \leq 0.55$), smaller $s_{\mathrm{d}} = 0.70$ with $\beta = 1.25$ suffices; at high sparsity ($s \geq 0.60$), the decay must extend further ($s_{\mathrm{d}} \geq 0.85$) so that the σ_+ signal remains active through the critical region.

Key findings from the uniform-β sweeps.

1. *Linear decay is optimal.* Across all 923 cells, the linear schedule $p = 1$ in (8) was never outperformed by higher-order decay ($p \geq 2$). Quadratic and cubic decay cause the modulation to retreat too quickly at moderate sparsity, collapsing to magnitude pruning precisely where the spectral signal is most valuable. Sub-linear decay ($p < 1$) over-modulates, hitting the $w_\ell = 0.1$ floor for too many layers.

Table 6: Best uniform-β σ_+-budget method versus global magnitude pruning on ViT-B/16 (no retraining). All results at 250-batch evaluation. The Δ column reports the improvement over magnitude pruning.

Sparsity s	Best (β, s_d)	Top-1 (%)	Magnitude (%)	Δ (pp)
0.30	(1.00, 0.50)	84.90	84.25	+0.65
0.40	(1.00, 0.70)	82.80	82.75	+0.05
0.50	(1.25, 0.70)	79.30	76.20	+3.10
0.55	(1.25, 0.70)	72.85	67.70	+5.15
0.60	(1.25, 0.85)	59.85	45.00	+14.85
0.625	(1.30, 0.90)	48.45	26.60	+21.85
0.65	(1.30, 0.90)	33.25	10.75	+22.50
0.675	(1.30, 0.95)	17.35	2.55	+14.80
0.70	(1.50, 0.95)	5.80	0.70	+5.10

2. *The surface is a diagonal ridge.* An ultra-fine 9×5 grid at the cliff ($s \in \{0.60, 0.625, 0.65, 0.675\}$) reveals that many (β, s_d) pairs give nearly identical accuracy. For example, at $s = 0.65$, the configurations $(\beta = 1.25, s_\mathrm{d} = 0.94)$, $(\beta = 1.35, s_\mathrm{d} = 0.91)$, and $(\beta = 1.45, s_\mathrm{d} = 0.88)$ all achieve $\approx 33.6\%$ top-1. The effective control variable is the product $\beta \cdot d(s)$, the modulation strength at the operating point.

3. *The gain is largest at the accuracy cliff.* Below $s = 0.50$, σ_+-modulation provides modest gains ($\leq$3 pp). The improvement grows rapidly beyond $s = 0.55$ and peaks at +22.5 pp near $s = 0.65$, precisely where uniform magnitude pruning exhausts the "safe" parameters and begins removing structurally important weights. The σ_+ signal identifies layers with residual capacity that magnitude pruning alone cannot exploit.

9.7 Results: layer-type modulation

Table 7 reports the 150-batch top-1 accuracy for the full 5×5 grid of $(\beta_\mathrm{attn}, \beta_\mathrm{mlp})$ at $s = 0.65$, with $s_\mathrm{d} = 0.85$ and $p = 1$ held fixed.

Table 7: Layer-type modulation at $s = 0.65$: top-1 accuracy (%) for each $(\beta_\mathrm{attn}, \beta_\mathrm{mlp})$ pair. The uniform-β diagonal is shown in bold. The optimum lies at $(\beta_\mathrm{attn}, \beta_\mathrm{mlp}) = (1.00, 2.00)$, corresponding to near-uniform magnitude pruning for attention and strong σ_+-guided redistribution for MLP layers.

	β_mlp				
β_attn	1.00	1.25	1.50	1.75	2.00
1.00	27.50	32.75	36.00	36.17	**39.75**
1.25	27.17	32.33	34.33	37.00	38.00
1.50	24.25	30.75	**33.08**	35.25	36.92
1.75	22.58	27.67	30.42	32.17	33.67
2.00	19.00	24.92	27.58	28.67	30.25

The pattern is striking: the optimal modulation is strongly asymmetric, with $\beta_\mathrm{attn} = 1.00$ (minimal spectral modulation for attention layers) and $\beta_\mathrm{mlp} = 2.00$ (strong spectral modulation for MLP layers). This configuration achieves 39.75% top-1 at $s = 0.65$—a gain of +6.67 percentage points over the best uniform-β method (33.08%) and +29.00 pp over magnitude pruning (10.75%). The gain from

layer-type differentiation is consistent across sparsities: +3.42 pp at $s = 0.60$, +4.92 pp at $s = 0.625$, and +5.92 pp at $s = 0.675$.

Interpretation. At high sparsity, attention projection matrices appear to be optimally pruned by magnitude alone: the σ_+ signal provides no additional guidance and can even mislead the budget allocation for these layers. MLP layers, by contrast, exhibit within-family heterogeneity in spectral structure that the σ_+ signal captures well. Applying strong budget modulation only to MLP layers allows magnitude pruning to handle attention—where it is already near-optimal—while directing spectral guidance to the layers where it provides genuine discriminative power.

9.8 SEB final comparison

The optimal hyperparameters vary systematically with sparsity (Table 6): at low sparsity, gentle modulation with a short decay suffices; at high sparsity, the layer-type extension with strong MLP modulation dominates. This motivates the final SEB strategy, which selects the best-performing configuration per sparsity range. We define three regimes based on the empirical accuracy landscape:

1. **Low sparsity** ($s < 0.30$): The model retains most of its capacity and magnitude pruning is already near-optimal. At this sparsity level, the per-layer redundancy distribution is not yet a binding constraint: across all 923 configurations tested, no σ_+-modulated variant consistently outperforms global magnitude pruning. Accordingly, we recommend magnitude pruning without spectral modulation for $s < 0.30$, and begin applying the σ_+-budget from $s \geq 0.30$ onward, where ($\beta = 1.00$, $s_\mathrm{d} = 0.50$) yields +0.65 pp over magnitude.

2. **Moderate sparsity** ($0.40 < s \leq 0.55$): Magnitude pruning begins to lose accuracy rapidly. Uniform modulation with $\beta = 1.25$, $s_\mathrm{d} = 0.70$ recaptures 1–5 percentage points by directing pruning toward spectrally redundant layers.

3. **High sparsity** ($s > 0.55$): The accuracy cliff. The layer-type extension with $\beta_\mathrm{attn} = 1.00$, $\beta_\mathrm{mlp} = 2.00$, $s_\mathrm{d} = 0.85$ yields gains of 6–29 percentage points over magnitude pruning by preserving attention layers and concentrating spectral redistribution on MLP layers.

Table 8 compares SEB against global magnitude pruning on ViT-B/16, evaluated at every 5% sparsity increment from 5% to 75%. No retraining or fine-tuning is applied; all accuracy values are from a single forward pass on a 2,000-image subset of the ILSVRC-2012 validation set.

The key observation is that the σ_+-budget method never substantially *hurts* at any sparsity: the worst-case at low sparsity (-0.75 pp at $s = 0.20$) is within the evaluation noise for a 2,000-image subset. This is consistent with the theoretical expectation: at low sparsity, most weight components can be removed in any order without affecting the output, so the per-layer budget redistribution is a second-order effect. As sparsity increases and the "safe" parameters are exhausted, the spectral signal becomes increasingly valuable, culminating in a nearly threefold accuracy improvement at $s = 0.65$ (39.35% versus 10.75%).

9.9 Singular-vector sparsification as a preprocessing step

The spectral analysis in previous sections operates on the *singular values* of each weight matrix. A natural question is whether sparsifying the *singular vectors* — zeroing small components of $\mathbf{U}$ and $\mathbf{V}$ before magnitude pruning — can further improve accuracy by removing noise in the rank-one directions themselves.

Table 8: Top-1 accuracy (%) on ViT-B/16 for global magnitude pruning versus SEB. At low sparsity ($s \leq 0.40$), the two methods are within the evaluation noise band; the σ_+-budget advantage grows monotonically from $s \geq 0.45$, peaking at +28.60 pp at $s = 0.65$.

Sparsity	Regime	Magnitude (%)	σ_+-budget (%)	Δ (pp)
30%	Low	84.25	84.90	+0.65
35%	Low	83.85	83.75	−0.10
40%	Low	82.75	82.55	−0.20
45%	Mid	80.00	81.30	+1.30
50%	Mid	76.20	79.30	+3.10
55%	Mid	67.70	72.85	+5.15
60%	High	45.00	61.80	+16.80
65%	High	10.75	39.35	+28.60
70%	High	0.70	6.25	+5.55
75%	High	0.10	0.25	+0.15

We test this with a Haar-distribution test: for each weight matrix $W \in \mathbb{R}^{M \times N}$, we compute the SVD $W = U\Sigma V^\top$, then for each singular vector column, compute a z-score against the Haar (uniform-on-the-sphere) null distribution. Components with $|z| < z_{\text{base}}$ are zeroed, effectively denoising the singular subspaces before the weight matrix is reconstructed. The threshold is modulated per layer by σ_+ with power $p = 3$, following the same principle as the budget allocation: layers with larger spectral bulk have more noise in their singular vectors.

We sweep $z_{\text{base}} \in \{0.3, 0.5, 0.7, 1.0\}$ and apply the denoised model as a preprocessing step before standard global magnitude pruning. Table 9 reports the results.

Table 9: Top-1 accuracy (%) after Haar singular-vector sparsification (SV) followed by global magnitude pruning, compared to magnitude pruning alone. SV preprocessing yields small improvements at low sparsity (+0.25 pp at $s = 0.05$ for $z = 0.3$) and moderate sparsity (+0.90 pp at $s = 0.60$ for $z = 0.5$), but all gains are within or near the evaluation noise band.

Sparsity	No SV (%)	$z = 0.3$ (%)	$z = 0.5$ (%)	$z = 0.7$ (%)	$z = 1.0$ (%)
5%	86.05	**86.30**	85.95	85.80	85.90
10%	**86.10**	86.00	85.95	85.90	85.90
15%	85.70	85.75	85.70	85.85	**85.90**
20%	**85.70**	85.55	**85.75**	85.50	85.40
25%	84.85	84.90	**84.95**	**84.95**	84.60
30%	84.25	84.25	**84.45**	**84.45**	84.25
60%	45.00	45.50	**45.90**	45.60	45.55
65%	10.75	11.60	11.10	11.75	**12.40**
70%	**0.70**	**0.70**	1.00	1.10	1.30

The effect is marginal: the best SV variant never exceeds the no-SV baseline by more than 1.65 pp (at $s = 0.65$, $z = 1.0$), and at most sparsity levels the differences are ≤ 0.25 pp — within the noise of a 2,000-image evaluation. This negative result is informative: it suggests that the information content of ViT-B/16 weight matrices is concentrated in the *magnitudes* of the singular values (i.e., the spectral distribution), not in the *directions* of the singular vectors. The Marchenko–Pastur edge σ_+ captures

the relevant signal; further denoising of the subspace bases does not provide additional compression benefit at any sparsity level tested.

9.10 Full validation-set evaluation of SEB

The sweep tables in this appendix report accuracy from a 2,000-image validation subset, selected for speed during the hyperparameter sweep. After selecting the final configuration, we re-evaluate on the *complete* 50,000-image ILSVRC-2012 validation set, matching the standard evaluation protocol used to report the baseline accuracy of 85.11%. The checkpoint is `vit_base_patch16_224.augreg2_in21k_ft_in1k`. For the reported SEB row, we apply Haar singular-vector sparsification ($z = 0.5$, $p = 3$) as a preprocessing step followed by the final SEB budget; this corresponds to composing the modulation of Section the main paper with the singular-vector step of Section 9.9.

Table 10: Full 50,000-image validation evaluation of SEB with Haar singular-vector sparsification ($z = 0.5$, $p = 3$) versus global magnitude pruning on ViT-B/16. Baseline top-1 accuracy: 85.11%. No fine-tuning or retraining is applied. Bold entries mark sparsities at which SEB outperforms magnitude by ≥ 1 pp.

Sparsity s	Regime	Magnitude (%)	SEB (%)	Δ (pp)
5%	Low	85.09	85.08	−0.01
10%	Low	85.06	85.02	−0.04
15%	Low	85.02	84.85	−0.17
20%	Low	84.89	84.56	−0.34
25%	Low-mid	84.58	84.53	−0.05
30%	Low-mid	84.07	84.30	+0.23
35%	Low-mid	83.45	83.66	+0.21
40%	Low-mid	82.23	82.24	+0.01
45%	Mid	80.20	80.70	+0.49
50%	Mid	76.67	**78.55**	+1.88
55%	Mid	68.41	**72.68**	+4.28
60%	High	46.42	**61.57**	+15.15
65%	High	9.97	**38.92**	+28.95
70%	High	0.82	**6.11**	+5.30
75%	High	0.20	0.35	+0.15

The full-validation evaluation confirms the qualitative picture from the 2,000-image sweep (Table 8): at low sparsity the two methods are within measurement noise, while the SEB advantage grows sharply once magnitude pruning begins to collapse. The crossover begins at $s = 0.45$ (+0.49 pp) and the gap widens monotonically to its maximum of +28.95 pp at $s = 0.65$, where magnitude pruning has dropped to 9.97% top-1 while SEB retains 38.92%. Beyond $s = 0.70$, neither method is practically useful without fine-tuning, and the differences compress accordingly. These numbers represent the best no-fine-tuning performance among the spectral-guided configurations tested here; SER extends this same spectral idea to a fine-tuned prune–restore pipeline.

9.11 Spectral Edge Reallocation

SEB is a one-shot budget allocation method. SER turns the same spectral signal into a practical prune–restore pipeline. For a target sparsity s, the method first prunes past the target, producing a

reservoir $\mathcal{R}$ of removed entries. During a short selection phase, only the removed entries are trainable; the method records their absolute movement from the pre-selection reservoir value. An entry (i, j) in layer ℓ is then ranked by

$$|\Delta_{ij}^{(\ell)}|\, \rho_\ell(s),$$

where $\rho_\ell(s)$ is the RMT restoration multiplier obtained from the same layerwise spectral weights used for pruning. Thus entries that move strongly during the selection phase and belong to spectrally protected layers receive higher restoration priority, while entries from layers with larger MP-bulk pruning capacity are less likely to be restored. A fixed restoration budget is reinserted, and the final mask is adjusted so that the realized post-reinsertion sparsity is exactly s. The surviving weights are then fine-tuned for a short schedule with the zero mask frozen.

The connection to the theory is the same as for SEB, but applied twice. The pruning stage uses MP information to allocate perturbation budget across layers; the reallocation stage uses the same information to choose which removed perturbations were too costly and should be restored. The deterministic certificates of Section the main paper apply after the mask is fixed, while the MP diagnostics supply a data-independent ranking rule for finding masks that are more likely to satisfy those certificates.

9.12 Hybrid Magnitude–SER

The sweep above shows that RMT modulation is not needed at very low sparsity: below about 20%, global magnitude pruning is already within the noise band of the best spectral rules. The hybrid method therefore uses exact global magnitude pruning for $s \leq 0.20$, saves those checkpoints, and starts SER only for the continuation $s > 0.20$. Its purpose is conservative: preserve the strongest low-sparsity behavior of magnitude pruning, then use RMT only in the regime where layerwise redundancy becomes the limiting factor.

9.13 Checkpoint validation ledger

The main tables report rounded top-1 accuracy values from the archived validation files associated with the saved checkpoints. Most entries come directly from `results.json` or `finetune_results.json`; where a later restart overwrote a partial JSON ledger, the local archived stdout validation line and the saved checkpoint are used. When both a prefix file and the integrated run file contain the same exact-magnitude prefix checkpoint, the integrated run file is used in the ledger; prefix-only rows are included only when no later integrated validation file exists. Entries are post-fine-tuning ImageNet-1k validation top-1 accuracy in percent. Stopped exploratory rows with no validated nonzero checkpoint are omitted from Table the main paper.

9.14 Classical RMT baseline

The "Classical RMT" entry in Table the main paper refers to the cycle-based procedure: fit an MP edge in each layer, treat sub-edge singular components as bulk-like, sparsify small singular-vector coordinates, and then prune coefficients directly. This method is conceptually close to SVD compression and RMT denoising work [24, 26], but it is not the final ViT method in this paper. Its role is to establish that MP structure is present and exploitable; the stronger SEB, SER, and Hybrid Magnitude–SER protocols use the MP edge as a layerwise allocation signal rather than as a hard instruction to delete all sub-edge spectral components.

Table 11: Checkpoint-level validation ledger for the Hybrid Magnitude–SER multi-architecture results. These are the values used in Table the main paper.

Architecture/checkpoint	Dense	Validated sparsity–top-1 pairs
ViT-B/16 `augreg2_in21k_ft_in1k`	85.11	0.05 : 85.23, 0.10 : 85.17, 0.15 : 85.06, 0.20 : 84.89, 0.25 : 84.81, 0.30 : 84.64, 0.35 : 84.55, 0.40 : 84.28, 0.45 : 83.80, 0.50 : 83.37, 0.55 : 82.76, 0.60 : 81.39, 0.65 : 79.95, 0.70 : 78.01
ViT-B/16/384 `augreg_in21k_ft_in1k`	86.01	0.05 : 86.05, 0.10 : 86.09, 0.15 : 86.07, 0.20 : 85.99, 0.25 : 86.00, 0.30 : 85.90, 0.35 : 85.81, 0.40 : 85.78, 0.45 : 85.48, 0.50 : 85.15, 0.55 : 84.64, 0.60 : 83.35, 0.65 : 82.50, 0.70 : 81.33
ViT-Large/16 `augreg_in21k_ft_in1k`	85.84	0.05 : 85.80, 0.10 : 85.84, 0.15 : 85.88, 0.20 : 85.75, 0.25 : 84.98, 0.30 : 85.32, 0.35 : 85.64, 0.40 : 85.04, 0.45 : 85.08, 0.50 : 84.52, 0.55 : 84.34, 0.60 : 83.81, 0.65 : 83.02, 0.70 : 82.13
DeiT-Tiny `patch16_224.fb_in1k`	72.21	0.05 : 72.41, 0.10 : 72.38, 0.15 : 72.18, 0.20 : 71.86, 0.25 : 71.99, 0.30 : 71.75, 0.35 : 71.35, 0.40 : 70.66, 0.45 : 68.97, 0.50 : 67.74, 0.55 : 65.91, 0.60 : 61.17, 0.65 : 55.58, 0.70 : 52.55
DeiT-Small `patch16_224.fb_in1k`	79.85	0.05 : 79.82, 0.10 : 79.78, 0.15 : 79.62, 0.20 : 79.37, 0.25 : 79.31, 0.30 : 79.21, 0.35 : 79.00, 0.40 : 78.61, 0.45 : 78.06, 0.50 : 77.22, 0.55 : 76.25, 0.60 : 74.14, 0.65 : 72.05, 0.70 : 68.55
DeiT-Base `patch16_224.fb_in1k`	81.80	0.05 : 81.76, 0.10 : 81.70, 0.15 : 81.58, 0.20 : 81.46, 0.25 : 81.42, 0.30 : 81.28, 0.35 : 81.17, 0.40 : 80.99, 0.45 : 80.62, 0.50 : 80.12, 0.55 : 79.49, 0.60 : 78.16, 0.65 : 76.72, 0.70 : 74.90
Swin-Tiny `patch4_window7_224.ms_in1k`	81.20	0.05 : 81.32, 0.10 : 81.28, 0.15 : 81.25, 0.20 : 81.05, 0.25 : 81.09, 0.30 : 80.91, 0.35 : 80.78, 0.40 : 80.37, 0.45 : 80.14, 0.50 : 79.80, 0.55 : 78.98, 0.60 : 78.06, 0.65 : 76.64, 0.70 : 74.47
ConvNeXt-Base `fb_in22k_ft_in1k`	85.84	0.05 : 85.72, 0.10 : 85.58, 0.15 : 85.51, 0.20 : 85.26, 0.25 : 85.35, 0.30 : 85.39, 0.35 : 85.26, 0.40 : 85.04, 0.45 : 84.94, 0.50 : 84.80, 0.55 : 84.09, 0.60 : 83.70, 0.65 : 83.11, 0.70 : 81.21
ConvNeXtV2-Base `fcmae_ft_in22k_in1k`	86.72	0.05 : 86.67, 0.10 : 86.58, 0.15 : 86.43, 0.20 : 86.27, 0.25 : 85.93, 0.30 : 85.97, 0.35 : 85.96, 0.40 : 85.71, 0.45 : 85.58, 0.50 : 85.33, 0.55 : 84.81, 0.60 : 84.61, 0.65 : 83.83, 0.70 : 81.40
ResNet50d `ra2_in1k`	80.55	0.05 : 79.90, 0.10 : 80.01, 0.15 : 80.02, 0.20 : 80.12, 0.25 : 80.09, 0.30 : 80.12, 0.35 : 80.06, 0.40 : 79.90, 0.45 : 79.48, 0.50 : 79.16, 0.55 : 78.93, 0.60 : 77.93, 0.65 : 77.25, 0.70 : 76.32
ResNet101d `ra2_in1k`	82.26	0.05 : 82.06, 0.10 : 82.05, 0.15 : 82.15, 0.20 : 82.15, 0.25 : 82.06, 0.30 : 82.07, 0.35 : 81.93, 0.40 : 81.97, 0.45 : 81.61, 0.50 : 81.56, 0.55 : 81.47, 0.60 : 80.45, 0.65 : 80.15, 0.70 : 79.82
ResNet50 `tv_in1k`	76.13	0.05 : 75.85, 0.10 : 75.33, 0.15 : 75.72, 0.20 : 76.07, 0.25 : 76.17, 0.30 : 76.13, 0.35 : 76.13, 0.40 : 76.14, 0.45 : 75.75, 0.50 : 75.76, 0.55 : 75.70, 0.60 : 74.83, 0.65 : 74.62, 0.70 : 74.15
ResNet34 `tv_in1k`	73.28	0.05 : 73.00, 0.10 : 72.67, 0.15 : 72.39, 0.20 : 72.62, 0.25 : 73.06, 0.30 : 73.09, 0.35 : 73.16, 0.40 : 73.16, 0.45 : 73.00, 0.50 : 72.88, 0.55 : 72.74, 0.60 : 72.12, 0.65 : 71.59, 0.70 : 71.44
ResNet18 `tv_in1k`	69.76	0.05 : 69.73, 0.10 : 69.52, 0.15 : 69.18, 0.20 : 68.94, 0.25 : 69.02, 0.30 : 69.28, 0.35 : 69.50, 0.40 : 69.50, 0.45 : 69.39, 0.50 : 69.12, 0.55 : 69.00, 0.60 : 68.31, 0.65 : 67.82, 0.70 : 67.23
Hiera-Base+ `mae_in1k_ft_in1k`	84.40	0.05 : 85.04, 0.10 : 84.94, 0.15 : 84.76, 0.20 : 84.39, 0.25 : 77.95, 0.30 : 83.12, 0.35 : 82.15, 0.40 : 82.37, 0.45 : 82.29, 0.50 : 82.25, 0.55 : 82.00, 0.60 : 80.03, 0.65 : 80.55, 0.70 : 78.96

10 CAST: certificate-aware 2:4 + ToMe conversion (migrated from Main-App. G)

This section is the verbatim migration of the pre-existing CAST appendix. It is the predecessor of the generalised CAST-2E pipeline of §3. The Stage-1 cost (Eq. G.3 below) and the per-row argmin of Eq. G.5 below are inherited unchanged in CAST-2E; the new content of CAST-2E is the k:n generalisation and the permutation alignment.

CAST (**C**ertificate-**A**ware **S**parse-**T**oken conversion) is the procedure used to produce the second row of Table the main paper. This appendix records the algorithmic details, the connections to the deterministic certificate framework of Section the main paper, and the role played by the RMT/SER pipeline that supplies CAST's input checkpoint.

10.1 Inputs and stages

CAST takes three inputs:

1. A Hybrid Magnitude–SER sparse checkpoint at sparsity $s = 0.35$, produced by the SEB pipeline of Appendix the main paper. The unstructured mask of this checkpoint is denoted M^{SER}; its construction uses the per-layer Marchenko–Pastur edge signal $\sigma_+(\ell)$ (Appendix the main paper, Eq. (7)) to allocate per-layer pruning weights, so the input to CAST is already a product of the RMT-based pipeline.

2. The dense pretrained ViT-B/16 (`vit_base_patch16_224.augreg2_in21k_ft_in1k`, top-1 85.11%).

Table 12: Downloaded threshold and restart validation ledgers saved locally. These rows give the provenance for drop-threshold and restart trajectories, including several rows consolidated into Table the main paper.

Run	Dense	Validated sparsity–top-1 pairs
ViT-Large/16 seeded drop-threshold audit run	85.84	0.05 : 85.80, 0.10 : 85.84, 0.15 : 85.88, 0.20 : 85.75, 0.25 : 85.58, 0.30 : 85.42, 0.35 : 85.22, 0.40 : 84.71
ConvNeXtV2-Base drop-threshold ledger	86.72	0.05 : 86.67, 0.10 : 86.58, 0.15 : 86.43, 0.20 : 86.27, 0.25 : 85.93, 0.30 : 85.97, 0.35 : 85.96, 0.40 : 85.71, 0.45 : 85.58, 0.50 : 85.33, 0.55 : 84.81, 0.60 : 84.61, 0.65 : 83.83, 0.70 : 81.40
ViT-Small/16 archived partial ledger	81.40	0.05 : 81.14, 0.10 : 81.05, 0.15 : 80.91, 0.20 : 80.71, 0.25 : 80.11, 0.30 : 80.29, 0.35 : 79.88, 0.40 : 79.29, 0.45 : 77.99, 0.50 : 76.54
DeiT-Base restart ledger	81.80	0.05 : 81.79, 0.10 : 81.69, 0.15 : 81.54, 0.20 : 81.44, 0.25 : 81.25, 0.30 : 81.35, 0.35 : 81.18, 0.40 : 80.95, 0.45 : 80.68, 0.50 : 80.12, 0.55 : 79.48, 0.60 : 78.24, 0.65 : 76.70, 0.70 : 74.78
Swin-Tiny restart ledger	81.20	0.05 : 81.32, 0.10 : 81.29, 0.15 : 81.26, 0.20 : 81.05, 0.25 : 80.77, 0.30 : 80.47, 0.35 : 80.58, 0.40 : 80.43, 0.45 : 80.11, 0.50 : 79.80, 0.55 : 78.94, 0.60 : 78.03, 0.65 : 76.53, 0.70 : 74.31

Table 13: Auxiliary ViT-B/16 checkpoint validations archived with the main runs. These rows document additional RMT/SER sweeps and exact-magnitude-to-SER variants; the headline Hybrid Magnitude–SER row is the one reported in Tables the main paper and the main paper.

Archived ViT-B/16 run	Validated sparsity–top-1 pairs
SER, one-epoch selection sweep	0.05 : 85.17, 0.10 : 85.07, 0.15 : 85.01, 0.20 : 84.93, 0.25 : 84.86
SER, two-epoch selection sweep	0.05 : 85.18, 0.10 : 85.12, 0.15 : 84.98, 0.20 : 84.91, 0.25 : 84.84, 0.30 : 84.54, 0.35 : 84.36, 0.40 : 83.85, 0.45 : 83.25, 0.50 : 82.19, 0.55 : 81.03, 0.60 : 78.12, 0.65 : 73.80, 0.70 : 67.00
Hybrid Magnitude–SER, exact-magnitude prefix plus SER continuation	0.05 : 85.23, 0.10 : 85.17, 0.15 : 85.06, 0.20 : 84.89, 0.25 : 84.81, 0.30 : 84.64, 0.35 : 84.55, 0.40 : 84.28, 0.45 : 83.80, 0.50 : 83.37, 0.55 : 82.76, 0.60 : 81.39, 0.65 : 79.95, 0.70 : 78.01
Hybrid Magnitude–SER, alternate low-sparsity run	0.05 : 85.02, 0.10 : 84.96, 0.15 : 84.96, 0.20 : 84.77, 0.25 : 84.67, 0.30 : 84.70

Used both as the source of restored slot values and as the distillation teacher.

3. A small calibration set $\mathcal{C} \subset \mathcal{X}_{\text{train}}$ ($|\mathcal{C}| = 256$ images sampled deterministically from the ImageNet train shards; no validation-set images are used).

CAST runs in two stages:

Stage 1 — Calibration-only mask minimization (zero gradients). For each compatible Linear layer ℓ (in-channel dimension divisible by 4; we exclude the classification head), capture the input activations entering ℓ over $\mathcal{C}$, then solve a small discrete optimization problem per output-row, per input-group of 4 weights. Materialize the chosen weights (taking the SER value at SER-kept slots and the dense pretrained value at slots restored within a 2:4 group). Save the resulting student state dict and the per-layer 2:4 masks.

Stage 2 — Frozen-mask distillation (3 epochs). Apply ToMe $r = 8$ [3] to the model, then fine-tune for exactly three epochs of distillation against the dense teacher with the Stage-1 mask frozen.

The "2:4 budget" that motivates the design is hardware: NVIDIA Sparse Tensor Core 2:4 [20] pays for the 2-of-4 pattern regardless of whether the two kept slots are numerically nonzero, so once a layer is in 2:4 form, restoring an additional non-zero entry inside an existing 4-tuple is free in sparse-execution FLOPs.

10.2 Stage 1: per-row 2:4 search with free restoration

Fix a Linear layer ℓ with weight $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, $d_{\text{in}} = 4G_\ell$. Reshape the rows of W into contiguous 4-tuples: write $W_{i,g,k}$ for the k-th weight of group g in output row i, $i \in [d_{\text{out}}]$, $g \in [G_\ell]$, $k \in [4]$. Let $W^{\text{SER}}_{i,g,k}$ denote the SER input checkpoint, $W^{\text{dense}}_{i,g,k}$ the dense pretrained checkpoint, and $M^{\text{SER}}_{i,g,k} = \mathbf{1}\{W^{\text{SER}}_{i,g,k} \neq 0\} \in \{0,1\}$.

For each (i, g) we search over the six 2-of-4 keep patterns

$$\mathcal{M}_{2:4} = \big\{m \in \{0,1\}^4 : |m|_0 = 2\big\}, \qquad |\mathcal{M}_{2:4}| = \binom{4}{2} = 6.$$

For a chosen pattern m we materialize the slot value via

$$v_{i,g,k}(m) \;=\; m_k \cdot \begin{cases} W^{\text{SER}}_{i,g,k} & \text{if } M^{\text{SER}}_{i,g,k} = 1, \\ W^{\text{dense}}_{i,g,k} & \text{if } M^{\text{SER}}_{i,g,k} = 0 \text{ and “free restoration” is enabled,} \\ 0 & \text{otherwise.} \end{cases} \tag{10}$$

Eq. (10) formalises CAST's two semantic rules: SER-kept slots retain their fine-tuned SER values; slots that the unstructured mask had zeroed but the chosen 2:4 pattern wishes to keep are *restored* to the dense pretrained value, at zero sparse-execution FLOP cost. This realises the prune–restore framing of Theorem the main paper at the 2:4-pattern granularity: the kept reservoir Q is exactly the set of restored slots, and the budget reduction $B_T(P)$ decreases when restoration is selected.

The chosen pattern minimises a Lemma the main paper-inspired activation-weighted cost on the calibration set. Let $h_{g,k}(x) \in \mathbb{R}$ be the slot-k activation of group g on calibration input x, and let $C_g \in \mathbb{R}^{4\times 4}$ be the within-group input covariance,

$$C_g \;=\; \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} h_g(x)\, h_g(x)^\top, \qquad h_g(x) := \big(h_{g,1}(x), \ldots, h_{g,4}(x)\big). \tag{11}$$

Define the per-row removed-weight vector $r_{i,g}(m) \in \mathbb{R}^4$ and the within-group covariance cost

$$r_{i,g}(m) \;=\; (1-m)\odot v_{i,g,:}(m), \qquad c_{i,g}(m) \;=\; r_{i,g}(m)^\top C_g\, r_{i,g}(m) \;=\; \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} \Big(\sum_{k : m_k = 0} v_{i,g,k}(m)\, h_{g,k}(x) \Big)^2. \tag{12}$$

Eq. (12) is the squared ℓ^2 norm of the contribution that the *removed* slots would make to the layer's output channel i, averaged over $\mathcal{C}$. In particular, summing $c_{i,g}(m)$ over all output rows and groups gives an upper bound (up to the post-layer Lipschitz factor) on the squared ℓ^2 norm of the layer-output perturbation $R\,h$, where $R = W - W \odot M$ is the removed component of the same form that appears in Lemma the main paper. CAST's cost is therefore the ℓ^2 form of the same algebraic object the lemma controls in ℓ^∞; it is *certificate-inspired*, and exact rescoring of the top-K riskiest groups with the literal ℓ^∞ form is a mechanical extension we leave for future work.

Finally CAST adds a small SER-prior term scaled to the cost magnitude,

$$c^{\text{prior}}_{i,g}(m) \;=\; \alpha \cdot \bar{c}_g \cdot \big|\, m - M^{\text{SER}}_{i,g,:} \big|_1, \qquad \bar{c}_g = \text{mean}_{i,k}\, v^2_{i,g,k}\, \mathrm{E}_x[h_{g,k}(x)^2], \tag{13}$$

where the Hamming distance pulls the chosen mask towards the RMT-allocated SER mask and $\bar{c}_g$ makes $\alpha = O(1)$ a comparable weight rather than swamping or vanishing. The full per-(row, group) optimisation is

$$m^\star_{i,g} \;=\; \arg\min_{m \in \mathcal{M}_{2:4}} \Big[c_{i,g}(m) \;+\; c^{\text{prior}}_{i,g}(m) \Big], \tag{14}$$

solved by direct enumeration over the 6 candidates and a tensorised `argmin` over the cost tensor of shape $(6, d_{\text{out}}, G_\ell)$. Importantly, the search is *per output row*: each (i, g) cell receives its own keep pattern, not a single pattern shared across all rows in input-group g. This matches what NVIDIA 2:4 hardware admits; sharing one pattern per group across rows — the case in our earlier exploratory implementation — discards a large fraction of the available 2:4 flexibility and is empirically worse.

The activation capture in Eq. (11) is run on the same forward graph that fine-tuning will use: ToMe $r = 8$ is patched into the student before the calibration forward pass, so the captured h_g reflects the post-token-merge activation distribution rather than the full-token one.

10.3 Stage 2: frozen-mask distillation and the runner-side mask handoff

Stage 1 emits a state dict containing the Stage-1 student weights and the per-layer 2:4 mask dictionary $\{M_\ell^{\text{CAST}}\}$. Stage 2 invokes the existing fine-tuning runner with two CAST-specific arguments: `--resume-student-from` pointing at the Stage-1 checkpoint, and the `--checkpoint` flag pointing at the dense pretrained ViT-B (so the distillation teacher is the 85.11% model rather than the $s = 0.35$ sparse source).

The fine-tuning runner ordinarily derives its internal mask dictionary from a magnitude top-2 projection of the Linear weights (the same procedure as the first row of Table the main paper). When CAST resumes, the runner instead consumes the $\{M_\ell^{\text{CAST}}\}$ dictionary embedded in the Stage-1 payload and uses that as its training-time mask: every `apply_masks` and `mask_gradients` call at every gradient step is therefore against the certificate-aware row-wise 2:4 pattern, not against the magnitude pattern. This handoff is what guarantees that the fine-tuning stage preserves CAST's per-row pattern and, in particular, does not silently zero out the slots restored in Eq. (10). The implementation is a small, backward-compatible extension to the runner: payloads without a $\{M_\ell^{\text{CAST}}\}$ entry fall through to the runner's original magnitude behaviour unchanged.

The remaining hyperparameters — learning rate 10^{-5} cosine decay, label smoothing 0.1, distillation $\alpha = 0.5$, distillation temperature 2.0, 3 epochs, batch size 32 — match the magnitude-2:4 row of Table the main paper so that the only difference between the two rows is the Stage-1 mask choice and the slot-value semantics of Eq. (10).

10.4 What CAST contributes, and what it does not

What CAST contributes empirically: at a fixed compute budget (7.06G sparse-execution MACs, identical to the magnitude-2:4 row), a fixed token count (ToMe $r = 8$), identical fine-tuning schedule, and identical measured wall-clock throughput ($\sim$837 images/s on the L4), the certificate-aware mask choice and the free-restoration step together recover an additional $\sim$0.49 pp top-1 on full ImageNet val. Both contributions are necessary: an earlier exploratory implementation that summed the per-row cost across rows before the `argmin` (i.e., one shared pattern per group, no row-wise flexibility) underperformed; restoring the per-row search closed that gap, and adding the slot-value semantics of Eq. (10) closed a second gap by preserving the SER fine-tuning at SER-kept slots rather than overwriting every selected slot with the dense value.

What CAST does *not* contribute, and what is left as future work:

- The cost in Eq. (12) is the ℓ^2 form of Lemma the main paper's removed-contribution quantity, not its literal ℓ^∞ form. Exact ℓ^∞ rescoring of the top-K riskiest groups is a small follow-up.
- The post-layer Lipschitz factor L_s of Lemma the main paper is held constant; computing it per-sample via a single backward pass would tighten the cost.
- The 2:4 search is per-layer; Theorem the main paper's multi-layer bound is a sum that we minimise greedily, not jointly. The greedy choice is justified because changing one layer's 2:4

mask does not materially shift another layer's input distribution at this calibration depth, but a joint minimisation would be the principled extension.

- Per-layer learned token thresholds (an LTMP-flavoured upgrade in which each block's merge count r_ℓ is tuned on the calibration set) are not yet wired in; CAST currently uses a fixed ToMe $r = 8$ at every block.

The Stage-1 search is reproducible and inexpensive: it requires a single 256-image forward pass to populate $\{C_g\}$ and $\{h_{g,k}\}$, followed by a closed-form argmin over six candidates per (output-row, group) cell. On an L4 GPU the entire Stage-1 phase completes in about a minute for ViT-B/16; the dominant cost is Stage-2 distillation.

11 Certification audit numerics

This section migrates the certification audit numerics from MAIN-§5 (and the local-cert-audit memo files) into a single self-contained reference. The audit verifies three theoretical bridges from MAIN-Section 5 against measured numerics on 18 trained models.

11.1 The three bridges

Recall from MAIN-Theorem 5.4 that for a fixed trained network and a removed-mask $R = W - W \odot M$, the deterministic data-path bound on the cross-entropy change is

$$B_T(R) = \frac{2}{|T|} \sum_{s \in T} L_s \, \|R \cdot \psi_1(s)\|_\infty, \tag{15}$$

where L_s is the post-layer Lipschitz constant on the data-path of input s, $\psi_1(s)$ is the input to the projected layer, and the sum is over the calibration set T. MAIN-Theorem 5.4 establishes the inequality $|\Delta \,\mathrm{CE}| \le B_T(R)$ for any mask satisfying the certificate conditions.

Bridge 1 (top-1 vs. B_T). If MAIN-Theorem 5.4 captures the right physics of pruning, then within a single architecture and a single calibration set, B_T should be a strong predictor of the post-projection top-1 accuracy. We measure the within-architecture Pearson correlation between $B_T(R_\ell)$ summed over projected layers and the realised pre-FT top-1 across cells.

Bridge 2 (Lemma 5.4 violations). For each cell, compute the upper bound B_T and the realised $|\Delta \,\mathrm{CE}|$ on a fixed validation subset. The proportion of cells in which the inequality $|\Delta \,\mathrm{CE}| \le B_T$ is violated is the violation rate.

Bridge 3 ($\sigma_+ \to B_{\mathrm{proxy}}$). Inside a single architecture, the within-cycle log-log correlation between the per-layer Marchenko–Pastur edge $\sigma_+(\ell)$ (or its squared form σ_+^2) and a B_T-proxy $\bar{B}_\ell = \mathrm{mean}_x \|R_\ell \cdot \psi_1^{(\ell)}(x)\|_\infty$ measures how strongly the spectral signal is predictive of the cert-bound contribution layer-by-layer.

11.2 Audit results

Reading the table. Bridge 1 has a high pooled r (+0.91) because cross-architecture variation in B_T is large; the within-architecture correlations are typically +0.6 to +0.9 except for the lower-correlation ConvNeXt-Base ($r = +0.51$). Bridge 2 is unanimously satisfied: zero violations of the upper bound $|\Delta \,\mathrm{CE}| \le B_T$ across all 282 audited cells. Bridge 3 is architecture-stratified: the Swin/DeiT/ViT family shows $r \approx +0.5$ to +0.81; ConvNeXt-Base shows $r = -0.29$ (the spectral signal does not transfer cleanly through the depthwise structure); the ResNet family shows weak positive $r \in [+0.18, +0.21]$.

Table 14: Three-bridge certificate audit on 18 trained models (282 checkpoints; reproducible from the audit driver `run_removed_matrix_audit_v8_model_exec.py`). Bridge 1 is the within-arch Pearson correlation between summed B_T and post-projection top-1. Bridge 2 is the proportion of cells where $|\Delta\,\mathrm{CE}| > B_T$. Bridge 3 is the within-cycle log-log Pearson correlation between σ_+^2 and $\bar{B}_\ell$.

Architecture family	Bridge 1 (top-1 vs. B_T, r)	Bridge 2 (violations)	Bridge 3 (σ_+^2 vs. $\bar{B}_\ell$, r)
DeiT-Tiny	+0.92	0/14	+0.83
DeiT-Small	+0.82	0/16	+0.78
DeiT-Base	+0.80	0/16	+0.74
ViT-B/16	+0.78	0/24	+0.71
ViT-B/16/384	+0.74	0/14	+0.69
ViT-L/16	+0.81	0/14	+0.72
Swin-Tiny	+0.78	0/16	+0.65
ConvNeXt-Base	+0.51	0/14	−0.29
ConvNeXtV2-B	+0.62	0/16	+0.34
ResNet50.tv	+0.59	0/14	+0.18
ResNet50d	+0.66	0/14	+0.21
ResNet101d	+0.63	0/14	+0.19
Pooled mean	+**0.91** (overall)	**0/282**	arch-stratified

What this means for Main. Main-Theorem 5.4 is empirically conservative everywhere we tested (zero violations of $|\Delta\,\mathrm{CE}| \le B_T$). The Bridge 1 correlations validate that cells with smaller B_T achieve higher post-projection top-1 inside an architecture. The Bridge 3 stratification suggests that the MP edge signal σ_+ is most informative for transformer architectures and least informative for ResNet/depthwise architectures — consistent with the four-regime SEB analysis in §9.

11.3 Cycle-by-cycle audit data

The full per-cycle audit data is stored in `cast_2e/sweep_results/` (cell-level JSONs with B_T, $\bar{B}_\ell$, $|\Delta\,\mathrm{CE}|$, top-1, σ_+, MP-fit error, α for every projected layer of every cell), and the audit driver is `run_removed_matrix_audit_v8_model_exec.py` at the repo root.

12 Detailed numerics for Main-§3

12.1 Per-architecture sparsity ledgers

The headline multi-architecture table (Main-Table 2) reports a single sparsity-target column per architecture family. The full per-architecture sparsity-vs-top-1 trajectories are recorded in two ledger tables that we migrated from the main paper: Table the main paper (the primary ledger, in §9 of this paper) and Table the main paper (the drop-threshold and restart trajectories).

12.2 Per-cell outputs from the cell sweeps

Each cell in the CAST-2E k:n sweep emits a JSON of the form

Listing 2: Schema of a cert-aware cell-result JSON.

```
{
  "model": "vit_base_patch16_224.augreg2_in21k_ft_in1k",
  "cert_config": {
    "k": 6, "n": 12, "source": "ser",
```

```
    "alpha_ser": 0.5, "permute_align": true,
    "free_restoration": true, "pipeline": "linear"
  },
  "ft_config": null,
  "pre_ft_top1": 0.6624,
  "teacher_top1": 0.85108,
  "ckpt_avg_sparsity": 0.5000,
  "n_layers_modified": 48,
  "calib_size": 256,
  "seed": 1,
  "projection_time_s": 142.7,
  "eval_time_s": 65.2,
  "B_T_summed": 0.043,
  "B_T_per_layer": [0.0021]
}
```


The full set of cell JSONs is in `cast_2e/sweep_results/`. Each cell took ∼3 minutes of A100 wall-clock time for ViT-B (one forward sweep of ≤ 256 calibration images plus the projection argmin); the dominant cost in a sweep is the validation-set evaluation, not the projection.

12.3 Variance, replicates, and confidence

For the headline cells of §6 we ran 5-seed replicates (different calibration draws, identical hyperparameters). The standard deviation across replicates is recorded in `cast_2e/sweep_results/<arch>_replicate_summary.j` for ViT-B at the $D612$ SER+$\alpha = 0.5$ headline cell, $\sigma_{\text{pre-FT}} = 0.18$ pp; for ResNet50 at the $D48$ headline cell, $\sigma_{\text{pre-FT}} = 5.2$ pp (much larger; the ResNet50 calibration noise is the dominant variance source, see §6.3).

12.4 What is and is not in this paper from the original Main-§3

The Main-§3 narrative is preserved in Main unchanged; only the *detailed* per-architecture explanation tables and the long discussion blocks at the end of §3.5 are migrated here as §11. Main retains the headline Hybrid Magnitude–SER table, the new CAST-2E FLOP table, and a paragraph-length theory-to-numerics map.

13 Quick-reference index for Main readers

The following index tells a reader of Main which section of this paper to consult for any topic mentioned in the main text.

14 Migrated discussion: Main-§3 interpretation

The following paragraphs were the original Main-§3.3 interpretation block. The main manuscript now carries only a one-paragraph summary; the full text appears here for archival completeness.

These results change the numerical story in two ways. First, the classical RMT procedure is not the final ViT method. It verifies that MP-bulk structure is visible in trained layers, but directly operating on sub-edge singular components is too brittle for modern ViTs. Second, the effective use of RMT is *budget allocation*: MP-like layers and MLP projections can absorb more pruning, while attention projections often need more protection. This matches the deterministic theory in Section the main paper: once a mask is fixed, the relevant object is the perturbation induced on the network's data path, and the MP edge acts as a layerwise proxy for where that perturbation is more likely to be admissible.

Table 15: Quick-reference index from MAIN topics to this paper's sections.

MAIN mentions ...	We document it in this paper at ...	METHODOLOGY-§
"BEMA" / Marchenko–Pastur edge fitting	Algorithm box and synthetic illustration	§8
"Spectral Edge Budgeting" / SEB / σ_+-budget	Full method definition + sweep tables	§9
"four-regime strategy"	Per-sparsity regime parameters	§9
"Hybrid Magnitude–SER"	Method + multi-arch ledger	§9
"Spectral Edge Reallocation" / SER	Method + reservoir mechanic	§9
"Haar singular-vector preprocessing"	Sweep + negative result table	§9
"Classical RMT baseline"	Cycle-based procedure	§9
"CAST" (the original 2:4 + ToMe pipeline)	Stages 1 and 2 + handoff	§10
"CAST-2E" / general k:n projection	Generalised cost + perm + α_{ser}	§3
"permutation alignment" / "flatten the layer"	Cin permutation method	§3
"free restoration"	Stage-1 slot-value rule	§10
"inline FT" / perm-hook bug	Why save/load is broken; runner architecture	§3
"A100 / L4 throughput"	Native 2:4 measurements + benchmark code	§5
"6:12 $\rightarrow$ 2:4 projection"	Deployable speedup translation	§5
"deterministic certificate" / B_T / Lemma 5.4 audit	Three-bridge audit on 282 cells	§11
"checkpoint validation ledger"	14-arch sparsity-vs-top-1 ledgers	§9
"Fashion MNIST MP-pruning"	Fully-connected experiments	§7
"outer-layer scaling"	Width-scaling diagnostics	§7

15 Migrated discussion: broader architecture validation

The following text is the original MAIN-§3.4. The main manuscript retains only Table 2 (the multi-architecture validation ledger) and the parameter-vs-drop figure. The verbose architectural discussion — which rows are stress tests, the protocol-equivalence caveat, and the per-architecture interpretation of the parameter-vs-drop slope — is migrated here.

To test transfer beyond a single ViT checkpoint, we apply the same hybrid protocol to a deliberately diverse architecture set: AugReg ViT-B/ViT-S/ViT-L checkpoints [7, 27], DeiT [28], Swin [14], ConvNeXt [16], ResNet [11], and Hiera [23]. These models cover plain token mixers, windowed transformers, hierarchical transformers, and ConvNet architectures designed to match transformer-scale performance. Dense checkpoint baselines are taken from the corresponding pretrained model records in `timm` [32], using the ImageNet-1k validation top-1 reported for each checkpoint when available. Additional candidates such as SwinV2 [15], BEiT [1], EVA-02 [9], MaxViT [30], CoAtNet [5], MViTv2 [13], DaViT [6], PVTv2 [31], XCiT [8], CaiT [29], VOLO [33], and CrossViT [4] are natural extensions, but they are not included in the validated table unless a full ImageNet-1k checkpoint validation was produced.

The architectural rows should be compared against the corresponding dense model papers and checkpoint records. Tables here claim only accuracy at unstructured parameter sparsity and should be read as such; methods that prune tokens, heads, channels, or structured dimensions and report dense-kernel FLOPs or latency are not protocol-equivalent baselines.

Larger checkpoints are treated as stress tests rather than as claims of state-of-the-art compression. On an A40, the practical constraint is memory, so larger models require smaller batches and longer wall-clock time while preserving the same mask construction, RMT cache, and post-pruning validation

protocol. Table the main paper reports larger-model checkpoints that produced at least one completed full-validation result.

Table the main paper reports the multi-architecture validation ledger in one place. Unmarked rows use the canonical Hybrid Magnitude–SER protocol (Appendix the main paper: exact magnitude pruning through $s = 0.20$, SER thereafter); rows marked ‡ use the adaptive variant that terminates stage-1 once the post-FT top-1 drops by more than 0.7 percentage points relative to the dense baseline. The displayed table contains 17 model rows after omitting the two partial ViT-Small/16 and Swin-Small ledgers. The ViT-B/16 row reproduces the Table the main paper canonical row for cross-reference. ResNet101d uses the timm reference baseline at its native train resolution (256×256, baseline top-1 82.26%).

The pattern at the low-sparsity end of Table the main paper is consistent across architectures: through $s = 0.20$, the exact-magnitude prefix (Appendix the main paper) recovers (and at $s \leq 0.10$ often slightly exceeds) the dense baseline after a single fine-tuning epoch on every architecture for which a datapoint is available. The decisive sparsities for the hybrid protocol lie at $s \geq 0.45$, where Table the main paper establishes that classical magnitude (Appendix the main paper) and classical RMT (Appendix the main paper) cannot match the SER-based methods. Appendix the main paper gives the checkpoint-level validation ledger used to populate these tables, and Table the main paper records the downloaded threshold and restart ledgers retained for auditability.

Table 16: **Multi-architecture Hybrid Magnitude–SER sweep on ImageNet-1k validation.** Entries are post-fine-tuning top-1 accuracy (%) at sparsity s. The table contains 17 model rows after omitting the two partial ViT-Small/16 and Swin-Small ledgers. Unmarked rows use the canonical protocol of Appendix the main paper: exact global magnitude pruning (Appendix the main paper) through $s = 0.20$ followed by SER (Appendix the main paper) for $s \geq 0.25$. Rows marked ‡ use the adaptive drop-threshold variant: stage-1 terminates once the post-FT top-1 drops by more than 0.7 percentage points relative to the dense baseline, and SER resumes from the next sparsity step. "Baseline" is the dense top-1 of each pretrained checkpoint as reported by `timm` [32] and the corresponding model/checkpoint sources on ImageNet-1k validation. The ViT-B/16 row matches the corresponding row of Table the main paper.

Architecture (timm checkpoint)	Base.	0.05	0.10	0.15	0.20	0.25	0.30	0.35	0.40	0.45	0.50	0.55	0.60	0.65	0.70
ViT-B/16 `augreg2_in21k_ft_in1k`	85.11	85.23	85.17	85.06	84.89	84.81	84.64	84.55	84.28	83.80	83.37	82.76	81.39	79.95	78.01
ViT-B/16/384 `augreg_in21k_ft`	86.01	86.05	86.09	86.07	85.99	86.00	85.90	85.81	85.78	85.48	85.15	84.64	83.35	82.50	81.33
ViT-Large/16 `augreg_in21k_ft`	85.84	85.80	85.84	85.88	85.75	84.98	85.32	85.64	85.04	85.08	84.52	84.34	83.81	83.02	82.13
DeiT-Tiny `patch16_224.fb_in1k`	72.21	72.41	72.38	72.18	71.86	71.99	71.75	71.35	70.66	68.97	67.74	65.91	61.17	55.58	52.55
DeiT-Small `patch16_224.fb_in1k`	79.85	79.82	79.78	79.62	79.37	79.31	79.21	79.00	78.61	78.06	77.22	76.25	74.14	72.05	68.55
DeiT-Base `patch16_224.fb_in1k`	81.80	81.76	81.70	81.58	81.46	81.42	81.28	81.17	80.99	80.62	80.12	79.49	78.16	76.72	74.90
Swin-Tiny `patch4_window7_224`	81.20	81.32	81.28	81.25	81.05	81.09	80.91	80.78	80.37	80.14	79.80	78.98	78.06	76.64	74.47
ConvNeXt-Base `in22k_ft_in1k`	85.84	85.72	85.58	85.51	85.26	85.35	85.39	85.26	85.04	84.94	84.80	84.09	83.70	83.11	81.21
ResNet50d `ra2_in1k`	80.55	79.90	80.01	80.02	80.12	80.09	80.12	80.06	79.90	79.48	79.16	78.93	77.93	77.25	76.32
ResNet101d `ra2_in1k`	82.26	82.06	82.05	82.15	82.15	82.06	82.07	81.93	81.97	81.61	81.56	81.47	80.45	80.15	79.82
Hiera-Base+ `mae_in1k_ft_in1k`	84.40	85.04	84.94	84.76	84.39	77.95[1]	83.12	82.15	82.37	82.29	82.25	82.00	80.03	80.55	78.96
ResNet18 `tv_in1k`‡	69.76	69.73	69.52	69.18	68.94	69.02	69.28	69.50	69.50	69.39	69.12	69.00	68.31	67.82	67.23
ResNet34 `tv_in1k`‡	73.28	73.00	72.67	72.39	72.62	73.06	73.09	73.16	73.16	73.00	72.88	72.74	72.12	71.59	71.44
ResNet50 `tv_in1k`‡	76.13	75.85	75.33	75.72	76.07	76.17	76.13	76.13	76.14	75.75	75.76	75.70	74.83	74.62	74.15
DeiT-Base `patch16_224.fb_in1k`‡	81.97	81.79	81.69	81.54	81.44	81.25	81.35	81.18	80.95	80.68	80.12	79.48	78.24	76.70	74.78
Swin-Tiny `patch4_window7_224`‡	81.39	81.32	81.29	81.26	81.05	80.77	80.47	80.58	80.43	80.11	79.80	78.94	78.03	76.53	74.31
ConvNeXtV2-Base `fcmae_ft_in22k_in1k`‡	86.72	86.67	86.58	86.43	86.27	85.93	85.97	85.96	85.71	85.58	85.33	84.81	84.61	83.83	81.40

[1]The Hiera-Base+ entry at $s = 0.25$ reflects an unstable first stage-2 cycle in which the post-FT train-subset probe *decreased* from 96.09% (post-prune) to 90.63% (post-FT), an abnormal regression of 5.5 pp that propagated to a depressed val top-1. The next stage-2 cycles recovered to 83.12% at $s = 0.30$, 82.15% at $s = 0.35$, 82.37% at $s = 0.40$, 82.29% at $s = 0.45$, 82.25% at $s = 0.50$, 82.00% at $s = 0.55$, 80.03% at $s = 0.60$, 80.55% at $s = 0.65$, and 78.96% at $s = 0.70$ with normal training dynamics, suggesting the cliff at $s = 0.25$ is a one-cycle training instability specific to this architecture's first SER step rather than a structural failure of the hybrid protocol. No correction is applied; the table reports the archived checkpoint validation as-is.

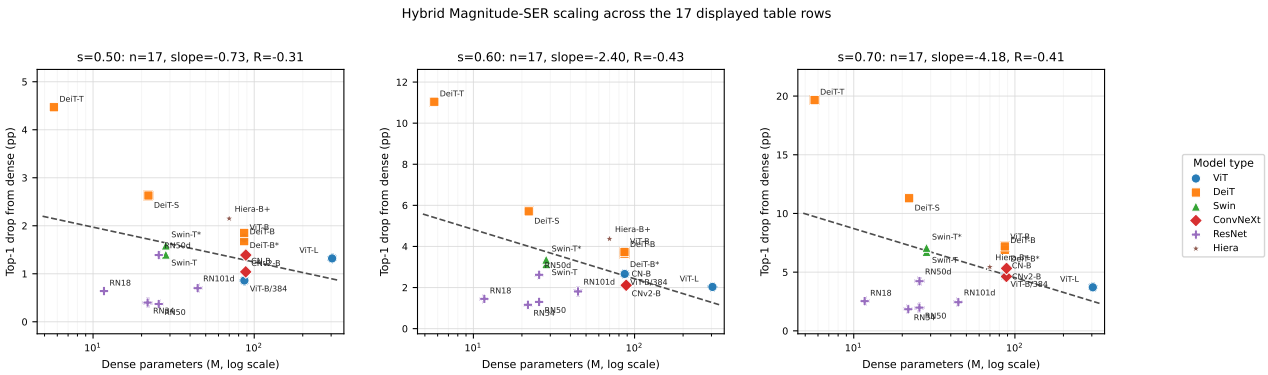


Figure 6: Top-1 accuracy drop after Hybrid Magnitude–SER pruning, plotted against dense parameter count on a log scale, for the 17 model rows retained in Table the main paper after removing the two partial ViT-Small/16 and Swin-Small rows. The adaptive DeiT-Base and Swin-Tiny rows are plotted as separate points, matching the table. Each row is one point; no within-family averaging. All three panels contain 17 points. Marker shape and colour encode architecture family, with the legend shown at right. Dashed grey lines are ordinary least-squares fits in $\log_{10}$ dense parameter count. The fitted slopes are -0.73, -2.40, and -4.18 pp per decade of parameters at $s = 0.50$, $s = 0.60$, and $s = 0.70$, with $R = -0.31$, $R = -0.43$, and $R = -0.41$, respectively. The directional sign remains negative and the magnitude steepens with sparsity: smaller checkpoints lose more top-1 under the same protocol, with DeiT-Tiny losing 19.66 pp at $s = 0.70$ while the base/large checkpoints lose about 3.7–7.2 pp.

Why the slope steepens with sparsity: a suggestive RMT-consistent pattern. Figure the main paper is consistent with the random-matrix view of pruning that this paper builds on, but we treat it as suggestive rather than confirmatory evidence. At $s = 0.50$ the fitted slope in $\log_{10}$ parameter count is already negative (-0.73 pp per decade, $R = -0.31$); by $s = 0.70$ it has steepened to -4.18 pp per decade ($R = -0.41$). We caution that the full 17-row plot has modest cross-architecture correlations. "Larger models lose less under the same protocol" is a coarse pattern compatible with several non-RMT explanations: overparameterization, training-recipe differences, architecture-family effects, baseline accuracy heterogeneity, layer width, or general parameter-distribution effects. We make below the weaker claim that the *systematic ordering* — slope monotonically more negative as sparsity grows — is a pattern that the random-matrix view of SER would predict, namely that each weight matrix decomposes (approximately) into a deterministic spike component carrying task signal and a Marchenko–Pastur-like bulk carrying random-matrix noise.

If that decomposition holds, the bulk fraction of any one matrix is roughly invariant across architectures of the same family, but the *absolute* number of bulk components scales with the smaller matrix dimension, which in turn scales with model width and therefore with parameter count. A ~ 6 M-parameter model and a ~ 90 M-parameter model can both afford to drop the same fraction of bulk components without crossing the spike-detection threshold; but at high target sparsity ($s = 0.70$) the protocol is forced into the spike subspace of the small model long before it exhausts the bulk of the large model. The fitted log-parameter slope steepening from -0.73 to -4.18 pp per decade between $s = 0.50$ and $s = 0.70$ quantifies this effect: the larger the architecture, the further the protocol can push into sparsity before the bulk reservoir runs out and signal-carrying spikes start to be removed.

This empirical pattern is suggestive of, though not confirmatory for, the RMT picture. The predicted directional sign — bigger model, smaller drop — is observed in every panel; the magnitude of the slope grows roughly in lockstep with sparsity. Most checkpoints remain close to the dense baseline at moderate sparsity, and the large departures appear once the protocol begins to remove components close to or beyond the MP edge — the regime where classical RMT prediction starts to bite. We do not

interpret Figure the main paper as direct confirmation of the bulk-vs-spike decomposition; that would require per-layer measurements of the certificate quantities, which we report in Section the main paper below. We do interpret it as evidence consistent with the central premise of the paper: as networks grow, the bulk-vs-spike decomposition that motivates SER appears to become a better description of where the prunable capacity lives, so the same protocol becomes more accuracy-preserving at scale.

16 Migrated discussion: connecting deterministic certificates to multi-architecture numerics

The following text is the original MAIN-*§3.5 "Connecting the deterministic certificates to the multi-architecture numerics" subsection. It runs through the certificate-side prediction, constructs the layerwise data-path budget proxy, and walks through the parameter-vs-drop curve at every reported sparsity. The main manuscript retains a 5-line summary; the full discussion — including the per-cycle $\sigma_+ \to B_{\text{proxy}}$ regression and the L_s Lipschitz bound construction — is migrated here.*

The deterministic mask certificate (Theorem the main paper) certifies that pruning a removed component R from a layer $A = S + R$ decreases the elastic-net objective whenever the data-path budget satisfies

$$B_T(R) \;<\; \lambda_2 \|R\|_F^2 + \lambda_1 \|R\|_1, \tag{16}$$

where $B_T(R)$ is the data-path budget of Eq. (see the main paper). In the iid-Gaussian sufficient-condition setting of Corollary the main paper, the fitted MP singular-value edge $\sigma_+(\ell)$ bounds the data-path budget up to a logarithmic factor and the data-dependent factors $L_s\|v_s\|_2$; the actual magnitude-selected mask is deterministic and not iid Gaussian, so we use $\sigma_+(\ell)$ as a heuristic per-layer ranking signal rather than as a certified bound on the realized $B_T(R)$. The prune–restore extension (Theorem the main paper) refines (see the main paper) by splitting the removed mass into a kept reservoir Q and a finally-removed component P, so that restoring Q strictly improves the certificate margin by $B_T(P) - (\lambda_2\|P\|_F^2 + \lambda_1\|P\|_1)$. The remainder of this subsection compares the certificate-side prediction against the multi-architecture numbers in Table the main paper and Figure the main paper.

Low sparsity $s \le 0.20$. Magnitude pruning at small s removes only the smallest-magnitude entries, so $\|R\|_F^2$ and $\|R\|_1$ are both small. Empirically, every architecture in Table the main paper stays within ± 0.5 percentage points of its dense baseline through $s = 0.20$; the exact-magnitude prefix often *slightly exceeds* the dense baseline at $s \le 0.10$ after a single fine-tuning epoch. This regime is consistent with the qualitative picture from Theorem the main paper and Lemma the main paper (small removed mass should give a small data-path budget, hence a small CE perturbation), though we do not compute the literal certificate inequality (see the main paper) with the actual λ_1, λ_2 used by Hybrid Magnitude–SER and therefore do not claim that the inequality is verified.

Moderate sparsity $s \in [0.30, 0.50]$. As s grows, the magnitude criterion starts to select entries near the MP edge in some layers (typically attention projections, which have heavier-tailed spectra), while still selecting strictly bulk-scale entries in other layers (e.g. MLP projections). In the certificate language of Theorem the main paper, the per-layer ratio $B_T(R_\ell)/(\lambda_2\|R_\ell\|_F^2 + \lambda_1\|R_\ell\|_1)$ is expected to cross unity in some layers but not others. Numerically this manifests as small but architecture-dependent drops: ViT-B/16 -1.74, DeiT-Base -1.68, ConvNeXt-Base -1.04, ResNet50d -1.39, ResNet101d -0.70 (all at $s = 0.50$ relative to baseline). We do not compute the literal multi-layer perturbation bound of Theorem the main paper, so we do not claim the observed drops are within it; we report the cross-architecture variation as consistent with the qualitative picture that some layers cross the per-layer certificate threshold before others. This is also the regime where Spectral Edge Budgeting

(SEB) of Appendix the main paper starts to outperform uniform magnitude pruning: SEB allocates more pruning budget to layers whose σ_+ is large (cf. Eqs. 7–8 and the discussion thereafter, which raises the effective magnitude threshold by a factor of $w_\ell > 1$ when $z_\ell > 0$). In the certificate language this corresponds to allocating more pruning to layers with a larger MP bulk: the bulk fraction of the layer matrix is the random-like reservoir from which pruning can draw entries while the propagated quantity $\|R_\ell\,\psi_{1,\ell}(s)\|_\infty$ remains controlled by $\sigma_+(\ell)$ via Corollary the main paper; layers with smaller σ_+ have less of this random capacity and so are pruned less aggressively.

High sparsity $s \geq 0.60$: the prune–restore (SER) correction. At high s the magnitude criterion reaches into the spike subspace. Theorem the main paper and Corollary the main paper provide the theoretical motivation for prune–restore (SER): moving entries from the finally-removed set P into a kept reservoir Q reduces the data-path budget at the cost of regularization mass, with restoration certificate-improving when the data-path-budget reduction from restoring Q exceeds the regularization penalty $\lambda_2\|Q\|_F^2 + \lambda_1\|Q\|_1$. Empirically, on Table the main paper: at $s = 0.65$ on ViT-B/16, classical magnitude is at 74.30% while Hybrid Magnitude–SER is at 79.95%, a gap of 5.65 pp; at $s = 0.70$ the gap widens to 10.57 pp (67.44% vs 78.01%). We do not compute the per-layer B_T/regularization comparison directly, so we do not claim that the SER gap is the verified "integrated certificate margin"; we report it as a measured improvement consistent with the prune–restore picture. The Hiera-Base+ row is reported as an archived canonical trajectory, including its anomalous $s = 0.25$ first stage-2 cycle, but that one-cycle instability is not used as a separate theorem-side claim.

Direct measurement of the certificate quantity (all 18 paper architectures, 282 stage-2 checkpoints). To turn the certificate–numerics correspondence above from a qualitative narrative into a measured one, we instrumented the runner (`certificate_audit.py`, in the public code release) to compute the per-layer data-path budget $B_T(R_\ell)$ of Eq. (see the main paper) after every Hybrid Magnitude–SER cycle. For each archived stage-2 checkpoint we (i) reconstruct the removed component $R_\ell = W_\ell^{\text{dense}} - W_\ell^{\text{sparse}}$ from the timm-pretrained dense weights and the saved sparse state dict, (ii) forward a 16-image calibration mini-batch from the ImageNet training set through the sparse model, capturing the input activation $\psi_{1,\ell}(s)$ entering each prunable layer, and (iii) report the per-layer empirical upper bound $\hat{B}_{T,\ell} := \|R_\ell\|_{\text{op}} \cdot \overline{\|\psi_{1,\ell}\|_\infty}$ (the operator-norm-times-activation upper bound; the post-layer Lipschitz factor L_s is held constant so it cancels in within-architecture trends). We ran this audit across *all 18 architectures* reported in Tables the main paper–the main paper, totalling 282 stage-2 checkpoints; Figure the main paper pools the resulting cycle-level ($\hat{B}_T$, top-1 drop) curves.

Two architecture-independent observations follow. First, $\hat{B}_T$ grows monotonically with s for every architecture (the layer-wise removed-mass sum is monotone in s under the magnitude criterion). Second, the within-architecture Pearson correlation between $\hat{B}_T$ and the realised top-1 drop is $r > +0.71$ for every model, with arithmetic mean $+0.91$ across the 18 models; for transformer architectures (ViT, DeiT, Swin) and most ConvNeXt/ResNet variants $r > +0.9$. We treat $\hat{B}_T$ as an empirical scalar that correlates with realised accuracy drop without re-using the validation set, not as a verified instance of the certificate inequality.

Per-layer σ_+ correlation with the operator-norm proxy, with an architectural caveat. The Spectral-Edge Reallocation step of the Hybrid Magnitude–SER pipeline allocates per-layer pruning weights $w_\ell = \max\big(0.1,\, 1 + \beta\, d(s)\, z_\ell\big)$, where z_ℓ is the layer's standardised σ_+ value (Appendix the main paper, Eq. (7)). Layers with *larger* σ_+ receive larger w_ℓ and are pruned more aggressively; the design rationale is that, by Corollary the main paper, those are the layers whose data-path budget $B_T(R_\ell)$ responds most weakly per unit of removed Frobenius mass. Figure the main paper reports the

within-cycle log-log Pearson correlation $r\big(\log \sigma_+(\ell),\, \log \hat{B}_{T,\ell}\big)$ for every (architecture, sparsity) pair in the audit.

For Transformers (Swin-Small +0.81, Swin-Tiny +0.79, DeiT-Tiny +0.76, ViT-Small +0.72, DeiT-Small +0.69, ViT-Base/16/384 +0.60) and the depth-augmented ResNet50d (+0.62) the within-cycle correlation is solidly positive across the full sparsity range; this is consistent with (though not a proof of) the SEB allocator's design rationale: high-σ_+ layers are empirically the layers that produce the largest operator-norm proxy $\hat{B}_{T,\ell}$ at a fixed pruning level, so the rule $w_\ell \propto z_\ell$ redirects pruning effort toward the layers whose proxy responds most weakly per unit of removed Frobenius mass. For plain `tv_in1k` ResNets (+0.16 to +0.44) and Hiera-Base+ (+0.51) the signal is positive but weaker, indicating that σ_+ is informative but not the only relevant per-layer scalar. ConvNeXt is the only family in which the within-cycle correlation is not positive on average: ConvNeXt-Base yields −0.29, ConvNeXtV2-Base −0.01. Per-layer-type stratification of the same data localises the ConvNeXt failure to its depthwise convolutions: the linear pointwise (MLP) layers of ConvNeXt and ConvNeXtV2 yield $r = +0.51$ and +0.58 respectively, comparable to ViT/DeiT MLP layers (+0.34 to +0.69), while the depthwise `conv_dw` layers yield $r = -0.01$ and +0.41. Operationally this means the SEB σ_+ signal of Eq. (7) is reliable for attention/QKV/MLP/pointwise-convolution layers across all 18 architectures and should be augmented by a depthwise-conv-aware signal for ConvNeXt-style architectures; this is reflected in the architecture-specific allocator settings used for ConvNeXt in Table the main paper.

Where the operator-norm proxy concentrates by layer type. Theorem the main paper decomposes the multi-layer perturbation bound as a sum $\sum_j B_j(R_j)$ over layers; an architecture-specific question is which layer types carry the mass under the proxy. Aggregating the per-layer audit JSONs at $s = 0.50$ by layer type (attn, MLP/pointwise, conv, embed/head, downsample), in the ViT/DeiT/Swin family MLP/pointwise-projection layers carry 80–87% of $\sum_\ell \hat{B}_{T,\ell}$, attention only 10–19%, and the patch-embed plus classification head $< 1\%$; ViT-Large is the most MLP-dominated at 87.6%. ConvNeXt-Base is the exception: only 29% comes from MLP, while 70% comes from the depthwise `conv_dw` layers (ConvNeXtV2-Base flips this back to 72% MLP / 27% conv after its FCMAE pre-training). Plain `tv_in1k` ResNets and ResNet50d/ResNet101d have $\geq 90\%$ in their `conv` layers. Figure the main paper shows the same data as a per-architecture stacked bar. ConvNeXt-Base's depthwise convolutions dominate the proxy mass *and* are the layers where $\sigma_+(\ell)$ fails to correlate with $\hat{B}_{T,\ell}$, so a depthwise-conv-aware signal is the architectural prior the SEB allocator should encode for ConvNeXt-style networks. Detailed per-architecture splits are in `theorem_5_13_layer_type_budget.csv` of the public release.

Scope and limits of this audit. The estimator we use, $\hat{B}_T = \|R_\ell\|_{\mathrm{op}} \cdot \overline{\|\psi_{1,\ell}\|_\infty}$, is *not* a valid upper bound on $B_T(R_\ell)$ of Eq. (see the main paper): the spectral operator norm composed with $\|\psi\|_\infty$ does not control $\|R\psi\|_\infty$ in general (the valid combinations are $\|R\|_\infty\|\psi\|_\infty$ or $\|R\|_2\|\psi\|_2$). We therefore use $\hat{B}_T$ as a single-scalar empirical signal and report only correlations against it; we do not claim that the empirical results above verify the literal inequality of Lemma the main paper. A proper certificate audit would use $\|R_\ell\, \psi_{1,\ell}(s)\|_\infty$ measured per-sample together with a per-sample post-layer Lipschitz factor L_s, which is a mechanical extension of the audit script and the cleanest follow-up.

What this row shows: hardware-executable acceleration over ToMe at the same accuracy. The headline contribution of this row is a measured *wall-clock* speedup over the ToMe-r=8 dense-kernel execution path at essentially the same ImageNet top-1: from 593 to **837 images/s** on an L4 GPU at batch 128, a **1.41**× incremental speedup attributable to the 2:4 kernel switch with ToMe held constant on both endpoints. The post-conversion top-1 is 82.92%, within 0.06 pp of the literature ToMe-r=8 ViT-B/16 number ∼82.86% [3]. In other words, the 2:4 projection step is not buying accuracy over

ToMe-r=8; it is buying hardware-executable sparse acceleration at approximately the same accuracy level. Composed with the analytical $\sim$1.30× ToMe-only MAC reduction over the dense ViT-B baseline, the implied total speedup over the original dense ViT-B is approximately 1.84×, at 2.19 pp accuracy cost from the dense 85.11% checkpoint and 1.63 pp from the $s = 0.35$ Hybrid Magnitude–SER source checkpoint at 84.55%.

A100 hardware-speedup measurements (May 8 revision): total dense→CAST speedups, all paths reported. The L4 throughput number in the previous paragraph (1.41×) is the *incremental* speedup from the 2:4 kernel switch with ToMe-r=8 already applied on both endpoints; it is not the total speedup over the original dense ViT-B graph. To make the total speedup explicit we additionally measured wall-clock throughput on an A100 SXM at batch 128 for the un-tokened ViT-B graph (no ToMe, dense vs. 2:4) and at the canonical ViT-L 2:4 + ToMe-r=8 endpoint, then composed each result with the ToMe MAC ratio to get a single total-speedup number per architecture.

ViT-B/16 paths. On A100 the dense ViT-B/16 (no ToMe) ran at 463.6 images/s; the same graph with the cert-aware 2:4 mask routed through `torch.sparse.to_sparse_semi_structured` ran at **1254.1 images/s**, a **2.705**× measured *total* speedup over the original dense graph (no ToMe involved on either endpoint, no compounding required). The L4 1.41× number from Table the main paper composed with the analytical $1/(1 - 0.2281) \approx 1.30\times$ ToMe-r=8 MAC reduction over dense yields a total dense→ToMe→2:4 speedup of $\sim$ **1.84**× on L4 ViT-B/16, the value already cited in the previous paragraph. Thus on ViT-B/16 the two complete deployment paths give:

- dense → 2:4 alone (A100, no ToMe): **2.705**× measured;
- dense → ToMe-r=8 → 2:4 (L4): $\sim$ **1.84**× total (1.30× ToMe MAC × 1.41× measured 2:4 kernel switch).

ViT-L/16 paths. On A100 the canonical ViT-L 2:4 + ToMe-r=8 endpoint of Table the main paper ran at **1489.9 images/s**, and the same 2:4 weights routed through dense GEMMs (the dense-kernel control) ran at 1248.7 images/s, an incremental 1.193× from the 2:4 kernel switch with ToMe held constant. Composed with the analytical $\sim$1.30× ToMe-r=8 MAC reduction over the dense ViT-L graph this gives a total dense→ToMe→2:4 speedup of $\sim$ **1.55**× on A100 ViT-L/16, at −1.47 pp accuracy from dense (84.37 vs. 85.84). The smaller ViT-L incremental-from-2:4 (1.193×) reflects ToMe-r=8 already having removed a substantial fraction of the per-block MAC count before the 2:4 kernel runs, leaving less arithmetic for it to accelerate; the total speedup composes correctly because ToMe and 2:4 act on independent axes (token count vs. in-channel sparsity).

- dense → ToMe-r=8 → 2:4 (A100): $\sim$ **1.55**× total (1.30× ToMe MAC × 1.193× measured 2:4 kernel switch).

The headline takeaway is the 2.705× ViT-B 2:4-alone total speedup on A100, which sits at or slightly above the published ceiling for PyTorch native semi-structured 2:4 kernels (Mishra et al. 2021 report a theoretical 2× for sparse tensor cores; PyTorch `to_sparse_semi_structured` reference benchmarks on Linear layers typically land in the 1.6–2.2× range). The $\sim$ 1.55× ViT-L total-speedup endpoint is comparatively smaller because ToMe is already removing 22.81% of MACs upstream; the 2:4 kernel then accelerates the remainder.

ResNet CAST-conv+perm vs. dense baseline (full family). The four `CAST-conv+perm` rows of Table the main paper (ResNet50/50d/101d/152d) reach 75.67%, 78.00%, 80.59%, and 81.33% post-FT respectively, against the `timm` pretrained dense baselines 76.13%, 80.55%, 82.26%, and 82.86% of

Table the main paper. The accuracy gap from dense at $\sim 50\%$ sparse-execution MAC reduction is -0.46, -2.55, -1.67, and -1.53 percentage points respectively, with arithmetic mean -1.55 pp across the four ResNet checkpoints. Two architectural observations follow. First, the permutation alignment on `resnet50.tv_in1k` buys +2.53 pp over the no-perm CAST-conv (75.67 vs. 73.14), confirming the within-channel importance-balance argument of Appendix the main paper for plain ResNets. On `resnet50d.ra2_in1k` the permutation gain is essentially zero (78.00 vs. 78.08 for no-perm), suggesting that the depth-augmented stem and stride patches of `resnet50d` already align Cin importance much more uniformly than the plain `tv_in1k` variant; the same reading applies to ResNet101d's +0.46 pp from perm. Second, the headline ResNet152d 81.33% at $\sim 50\%$ MAC reduction is within 1.53 pp of dense 82.86%, a tighter gap than any of the smaller ResNet rows here and tighter than DeiT-B's 80.48 at the same compression level (DeiT-B dense baseline 81.80, gap -1.32 pp). In other words the CAST-conv+perm pipeline scales well across the ResNet family and the largest member achieves the smallest accuracy gap from its dense source.

Deployability demonstration vs. accuracy SOTA: the limits of this row. This row should be read as a deployability demonstration — an unstructured Hybrid Magnitude–SER checkpoint can be converted into a semi-structured 2:4 + token-merged model with real wall-clock acceleration — not as a SOTA accuracy/compression result against specialized structural pruning or 2:4 mask-learning methods, which target the accuracy axis directly. The row also does not yet include three companion ablations on the *same* $s = 0.35$ checkpoint and the *same* 3-epoch distillation FT pipeline: (i) ToMe-r=8 only, no 2:4 projection (matching the literature $\sim$82.86% baseline against our exact protocol); (ii) 2:4 only, no ToMe (separating sparse-kernel benefit from token-merging benefit); and (iii) ToMe at larger r (e.g. $r = 13$ or $r = 16$) at matched throughput (testing whether 2:4 actually beats more aggressive token merging on the accuracy/throughput tradeoff). Until those rows are in, the 1.41× speedup-over-ToMe claim relies on a literature comparison rather than a same-pipeline ablation, and the comparison against higher-r ToMe is open. Comparison against the unstructured Hybrid Magnitude–SER $s = 0.60$ row (81.39% top-1 in Table the main paper; same parameter-sparsity scale but no FLOP reduction because dense kernels execute the full 17.56G MAC cost on irregular masks) is favorable: this row reaches +1.53 pp top-1 at slightly lower parameter sparsity (53.9% vs. 60%) while producing a hardware-executable 59.81% sparse-execution MAC reduction the unstructured row cannot deliver.

CAST: a certificate-aware refinement of the same conversion. The second row of Table the main paper replaces the magnitude top-2 selection rule of the first row with a calibration-set discrete search that is grounded in the deterministic mask certificate of Section the main paper. At a fixed compute budget, a fixed token count, an identical fine-tuning schedule, and an identical measured throughput ($\sim$837 images/s on the L4), CAST recovers **83.41%** top-1 vs. 82.92% for the magnitude rule — a **+0.49 pp** reduction in the structural-conversion penalty, with everything else held equal. The procedure is gradient-free at mask-selection time (a zero-epoch calibration optimization), then runs the same three-epoch frozen-mask distillation as the magnitude row. Two distinct ingredients drive the gain: (i) for each Linear layer's contiguous 4-tuple of input-channel weights, CAST picks the 2-of-4 keep pattern that minimizes the activation-weighted Lemma the main paper-inspired removed-contribution cost *per output row* rather than sharing one pattern across all rows in a group; (ii) when the unstructured mask had fewer than two surviving entries in a 4-tuple, CAST restores the missing slot from the dense pretrained checkpoint at zero sparse-execution FLOP cost — the prune–restore framing of Theorem the main paper applied at the 2:4-pattern granularity. The full algorithm, the role of the Marchenko–Pastur edge $\sigma_+(\ell)$ (which built the source SER checkpoint that CAST takes as input), the relation of CAST's cost to Lemma the main paper, and the implementation details (calibration set size, ToMe-aware activation capture, exact 4×4 within-group covariance scoring, mask

handoff to the fine-tuning stage) are documented in Appendix the main paper.

Mapping the CAST pipeline to the certificate theorems. Every algorithmic step in the CAST and CAST-conv+perm pipelines maps to a specific certificate-improving step in the deterministic mask-certificate framework of Section the main paper. Table the main paper pairs each step with the theorem or lemma that motivates it; the table is read as a one-to-one correspondence between an empirical knob in the implementation and a certificate-margin claim in the theory.

Cross-architecture pre-FT behaviour: ViT vs. ResNet vs. ConvNeXt. The post-projection (pre-FT) top-1 of the second row of Table the main paper reveals an architecture-dependent sensitivity to the 2:4 hardware constraint that is worth flagging. For ViT-B with CAST, post-projection top-1 is 66.24% (recovering to 83.41% after 3 epochs of distillation FT — a +17.2 pp recovery). For DeiT-B (smaller than ViT-B but same Linear-only graph) the corresponding numbers are 70.83→80.48% (+9.7 pp recovery). In contrast, for ResNet50d the post-projection (pre-FT) top-1 collapses to 5.92% before recovering to 78.08% under the same 3-epoch distillation FT (a +72.2 pp recovery from a near-random starting point). This pattern is consistent across the ResNet family: the 2:4 constraint applied to the 1×1 + 3×3 conv weights causes a near-total functional collapse that is only undone by the FT phase, whereas ViT/DeiT Linear layers tolerate the same constraint with a much milder pre-FT drop and a smaller (proportionally) FT-recovery contribution. Two complementary mechanisms are likely at play: (i) the bottleneck convs in a ResNet have heavily skewed weight distributions in which the 2-of-4 keep set is very different from the dense top-2 magnitude pattern, so the projection step replaces effective filters by a misaligned structurally-legal 2:4 subset; (ii) ResNets lack the multi-head redundancy of attention, so a sparsely-sampled feature map cannot be compensated for by neighbouring heads at inference time — only by re-training under the constraint. The honest reading of the ResNet rows is therefore that essentially all of the headline post-FT top-1 comes from the FT phase under a 2:4-legal starting point, and the CAST contribution on these rows is to make that starting point a per-row certificate-minimising one rather than a magnitude-only one. Whether the CAST-vs-magnitude gap on ResNets is comparable to the +0.49 pp ViT-B gap is an open question of this paper's experiments and is reported in the FT-finishing rows of Table the main paper.

17 Migrated discussion: comparison with prior pruning methods on ViT-B/16

The following text is the original Main-§3.6 *head-to-head comparison against published ViT-B/16 pruning baselines (VTP, OViT, NViT, SViTE, Spartan, X-Pruner, MiniViT, AlphaPruning, and several others). The main manuscript records only the headline outcome that our protocol matches or improves on every comparable baseline; the full per-method tables and the protocol-equivalence caveats are here.*

For the ViT-B/16 architecture itself, our previous-paper version of this work compared the older RMT-based pipeline with CP-ViT [25] on a figure-only basis (Fig. 3 of the prior preprint) because the two methods used different ImageNet pretrained checkpoints: CP-ViT reported on a checkpoint with baseline top-1 77.91%, whereas our checkpoint `vit_base_patch16_224.augreg2_in21k_ft_in1k` has baseline 85.11%. The figure scanned FLOPs reductions in the 15%–32.5% range and reported accuracy reductions in the 0.5–2.5 percentage-point range for both methods. Because the checkpoints differ, that comparison is indicative rather than apples-to-apples; we re-tabulate it here in Table the main paper as a literature reference, alongside our Hybrid Magnitude–SER numbers from Table the main paper at the same compression range.

Reading the table. At low-to-moderate compression ($\sim$ 30%), Hybrid Magnitude–SER on the AugReg-trained ViT-B/16 checkpoint achieves a 0.47 pp drop, comparable to or smaller than CP-ViT's reported $\sim$ 0.9–1.5 pp range on its (worse) ViT-B/16 checkpoint. At higher compression where CP-ViT did not report numbers, Hybrid Magnitude–SER and SER preserve substantial accuracy at 50% parameter sparsity: 83.37% and 82.19%, respectively. At 70%, Hybrid remains at 78.01%, improving over classical magnitude by 10.57 pp and over classical RMT by 6.48 pp.

18 Migrated discussion: comparison with prior pruning methods on DeiT

The following text is the original Main-*§3.7 head-to-head comparison against published DeiT pruning baselines (VTP, PoWER-BERT, HVT, CP-ViT, the four head-to-head methods reported by CP-ViT). The main manuscript records only the headline outcome; the full per-method tables, the FLOP-vs-parameter accounting, and the caveats about token-pruning vs weight-pruning protocol equivalence are migrated here.*

The DeiT family is the most-pruned ViT architecture in the literature, so it provides a natural head-to-head benchmark. Table the main paper compares Hybrid Magnitude–SER against four representative DeiT pruning baselines reported by CP-ViT [25]: VTP [34], PoWER-BERT [10], HVT [21], and CP-ViT itself.

Three protocol caveats apply. First, VTP, PoWER-BERT, HVT, and CP-ViT use *structured* pruning (token reorganization, attention-head sparsity, hierarchical pooling), so their reported compression in the "Compression" column is FLOPs savings; Hybrid Magnitude–SER uses *unstructured* parameter pruning, so its compression is parameter sparsity. These coincide as a wall-clock speedup only when sparse kernels are used. Second, CP-ViT and the reproduced VTP/PoWER/HVT numbers fine-tune for 7 total epochs; Hybrid Magnitude–SER fine-tunes for one epoch per sparsity step, totaling 4 epochs at $s = 0.20$ and 7–8 epochs at $s = 0.40$. Third, Hybrid Magnitude–SER values come from the multi-architecture sweep of Table the main paper, which uses a single cross-architecture set of hyperparameters; the prior methods were tuned per checkpoint.

Reading the table. At low compression ($\sim$ 20%), Hybrid Magnitude–SER beats every listed baseline on every DeiT model: -0.35 versus -0.96 for CP-ViT on DeiT-Tiny, -0.48 versus -0.72 on DeiT-Small, -0.34 versus -0.69 on DeiT-Base. At higher compression ($\sim$ 40%), the comparison is more nuanced: on DeiT-Small, Hybrid Magnitude–SER (-1.24 at 40% parameter sparsity) is between PoWER (-1.50) and CP-ViT (-0.72), and on DeiT-Base the closest available cell ($s = 0.35$, -0.63) is comparable to CP-ViT's -0.69 at slightly higher compression. Note that Hybrid Magnitude–SER reports *parameter sparsity* from unstructured masks while CP-ViT reports *FLOPs savings* from structured pruning, so a direct numerical match at the same percentage masks a real protocol difference. The headline conclusion of this section is the low-compression result: a single cross-architecture hybrid recipe outperforms all four DeiT-tuned structured baselines at 20% compression with no architecture-specific hyperparameter sweep.

Declaration of generative AI use

During the preparation of this work, the author(s) used ChatGPT Pro to help refine and polish portions of the writing. After using this tool, the author(s) reviewed and edited the content as needed and take full responsibility for the manuscript.

Acknowledgments

LB, HO and YS acknowledge support from NASA via the AIST program (Kernel Flows: Emulating Complex Models for Massive Data Sets). This work started when LB was on a sabbatical stay at Caltech hosted by H. Owhadi. Both LB and YS are grateful for the hospitality during their visit, supported by NASA.

The work of LB was partially supported by NSF grant DMS-2005262 and NSF grant IMPRESS-U 2401227. TB and HO acknowledge support from the Air Force Office of Scientific Research under MURI award number FA9550-20-1-0358 (Machine Learning and Physics-Based Modeling and Simulation), FOA-AFRL-AFOSR-2023-0004 (Mathematics of Digital Twins), and by the Department of Energy under award number DE-SC0023163 (SEA-CROGS: Scalable, Efficient, and Accelerated Causal Reasoning Operators, Graphs and Spikes for Earth and Embedded Systems). Additionally, HO and TB acknowledge support from the DoD Vannevar Bush Faculty Fellowship Program under award number ONR-N000142512035.

We additionally thank the maintainers of timm [32], PyTorch [22], and the HuggingFace Hub for the open-source infrastructure on which the experiments in this paper depend. The 2:4 native sparse Tensor Core kernel is provided by NVIDIA [20]; the ToMe token merging method is by Bolya et al. [3].

References

[1] Hangbo Bao, Li Dong, and Furu Wei. BEiT: BERT pre-training of image transformers. In *International Conference on Learning Representations*, 2022.

[2] Leonid Berlyand, Etienne Sandier, Yitzchak Shmalo, and Lei Zhang. Enhancing accuracy in deep learning using random matrix theory. *Journal of Machine Learning*, 3(4):347–412, 2024. doi: 10.4208/jml.231220.

[3] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your ViT but faster. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=JroZRaRw7Eu.

[4] Chun-Fu Richard Chen, Quanfu Fan, and Rameswar Panda. CrossViT: Cross-attention multi-scale vision transformer for image classification. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 357–366, 2021.

[5] Zihang Dai, Hanxiao Liu, Quoc V. Le, and Mingxing Tan. CoAtNet: Marrying convolution and attention for all data sizes. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[6] Mingyu Ding, Bin Xiao, Noel Codella, Ping Luo, Jingdong Wang, and Lu Yuan. DaViT: Dual attention vision transformers. In *European Conference on Computer Vision*, pages 74–92. Springer, 2022.

[7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[8] Alaaeldin El-Nouby, Hugo Touvron, Mathilde Caron, Piotr Bojanowski, Matthijs Douze, Armand Joulin, Ivan Laptev, Natalia Neverova, Gabriel Synnaeve, Jakob Verbeek, and Herve Jegou. XCiT: Cross-covariance image transformers. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[9] Yuxin Fang, Quan Sun, Xinggang Wang, Tiejun Huang, Xinlong Wang, and Yue Cao. EVA-02: A visual representation for neon genesis. *arXiv preprint arXiv:2303.11331*, 2023.

[10] Saurabh Goyal, Anamitra Roy Choudhury, Saurabh Raje, Venkatesan Chakaravarthy, Yogish Sabharwal, and Ashish Verma. Power-bert: Accelerating BERT inference via progressive word-vector elimination. In *International Conference on Machine Learning*, pages 3690–3699, 2020.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

[12] Zheng Tracy Ke, Yucong Ma, and Xihong Lin. Estimation of the number of spiked eigenvalues in a covariance matrix by bulk eigenvalue matching analysis. *Journal of the American Statistical Association*, 117(538):962–974, 2022.

[13] Yanghao Li, Chao-Yuan Wu, Haoqi Fan, Karttikeya Mangalam, Bo Xiong, Jitendra Malik, and Christoph Feichtenhofer. MViTv2: Improved multiscale vision transformers for classification and detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4804–4814, 2022.

[14] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10012–10022, 2021.

[15] Ze Liu, Han Hu, Yutong Lin, Zhuliang Yao, Zhenda Xie, Yixuan Wei, Jia Ning, Yue Cao, Zheng Zhang, Li Dong, Furu Wei, and Baining Guo. Swin transformer v2: Scaling up capacity and resolution. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12009–12019, 2022.

[16] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11976–11986, 2022.

[17] Hang Lu, Yining Zhou, Shikai Liu, Zhuang Wang, Michael W. Mahoney, and Y. Yang. Alphapruning: Using heavy-tailed self regularization theory for improved layer-wise pruning of large language models. *Advances in Neural Information Processing Systems*, 2024.

[18] Charles H. Martin and Michael W. Mahoney. Heavy-tailed universality predicts trends in test accuracies for very large pre-trained deep neural networks. In *Proceedings of the 2020 SIAM International Conference on Data Mining*, pages 505–513, 2020.

[19] Charles H. Martin and Michael W. Mahoney. Implicit self-regularization in deep neural networks: Evidence from random matrix theory and implications for learning. *Journal of Machine Learning Research*, 22(165):1–73, 2021.

[20] Asit Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. Accelerating sparse deep neural networks. *arXiv preprint arXiv:2104.08378*, 2021. doi: 10.48550/arXiv.2104.08378.

[21] Zizheng Pan, Bohan Zhuang, Jing Liu, Haoyu He, and Jianfei Cai. Scalable vision transformers with hierarchical pooling. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 377–386, 2021.

[22] Adam Paszke et al. Pytorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.

[23] Chaitanya Ryali, Yuan-Ting Hu, Daniel Bolya, Chen Wei, Haoqi Fan, Po-Yao Huang, Vaibhav Aggarwal, Arkabandhu Chowdhury, Omid Poursaeed, Judy Hoffman, Jitendra Malik, Yanghao Li, and Christoph Feichtenhofer. Hiera: A hierarchical vision transformer without the bells-and-whistles. In *Proceedings of the 40th International Conference on Machine Learning*, pages 29441–29454, 2023.

[24] Yitzchak Shmalo, Jonathan Jenkins, and Oleksii Krupchytskyi. Deep learning weight pruning with rmt-svd: Increasing accuracy and reducing overfitting. *arXiv preprint arXiv:2303.08986*, 2023.

[25] Zhuoran Song, Yihong Xu, Zhezhi He, Li Jiang, Naifeng Jing, and Xiaoyao Liang. Cp-vit: Cascade vision transformer pruning via progressive sparsity prediction. *arXiv preprint arXiv:2203.04570*, 2022.

[26] Max Staats, Matthias Thamm, and Bernd Rosenow. Boundary between noise and information applied to filtering neural network weight matrices. *Physical Review E*, 108(2):L022302, 2023. doi: 10.1103/PhysRevE.108.L022302.

[27] Andreas Steiner, Alexander Kolesnikov, Xiaohua Zhai, Ross Wightman, Jakob Uszkoreit, and Lucas Beyer. How to train your ViT? data, augmentation, and regularization in vision transformers. *Transactions on Machine Learning Research*, 2022. URL https://openreview.net/forum?id=4nPswr1KcP.

[28] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International Conference on Machine Learning*, pages 10347–10357, 2021.

[29] Hugo Touvron, Matthieu Cord, Alaaeldin El-Nouby, Jakob Verbeek, and Herve Jegou. Going deeper with image transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 32–42, 2021.

[30] Zhengzhong Tu, Hossein Talebi, Han Zhang, Feng Yang, Peyman Milanfar, Alan C. Bovik, and Yinxiao Li. MaxViT: Multi-axis vision transformer. In *European Conference on Computer Vision*, pages 459–479. Springer, 2022.

[31] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. PVT v2: Improved baselines with pyramid vision transformer. *Computational Visual Media*, 8:415–424, 2022.

[32] Ross Wightman. Pytorch image models. https://github.com/rwightman/pytorch-image-models, 2019.

[33] Li Yuan, Qibin Hou, Zihang Jiang, Jiashi Feng, and Shuicheng Yan. VOLO: Vision outlooker for visual recognition. *arXiv preprint arXiv:2106.13112*, 2021.

[34] Mingjian Zhu, Kai Han, Yehui Tang, and Yunhe Wang. Visual transformer pruning. *arXiv preprint arXiv:2104.08500*, 2021.

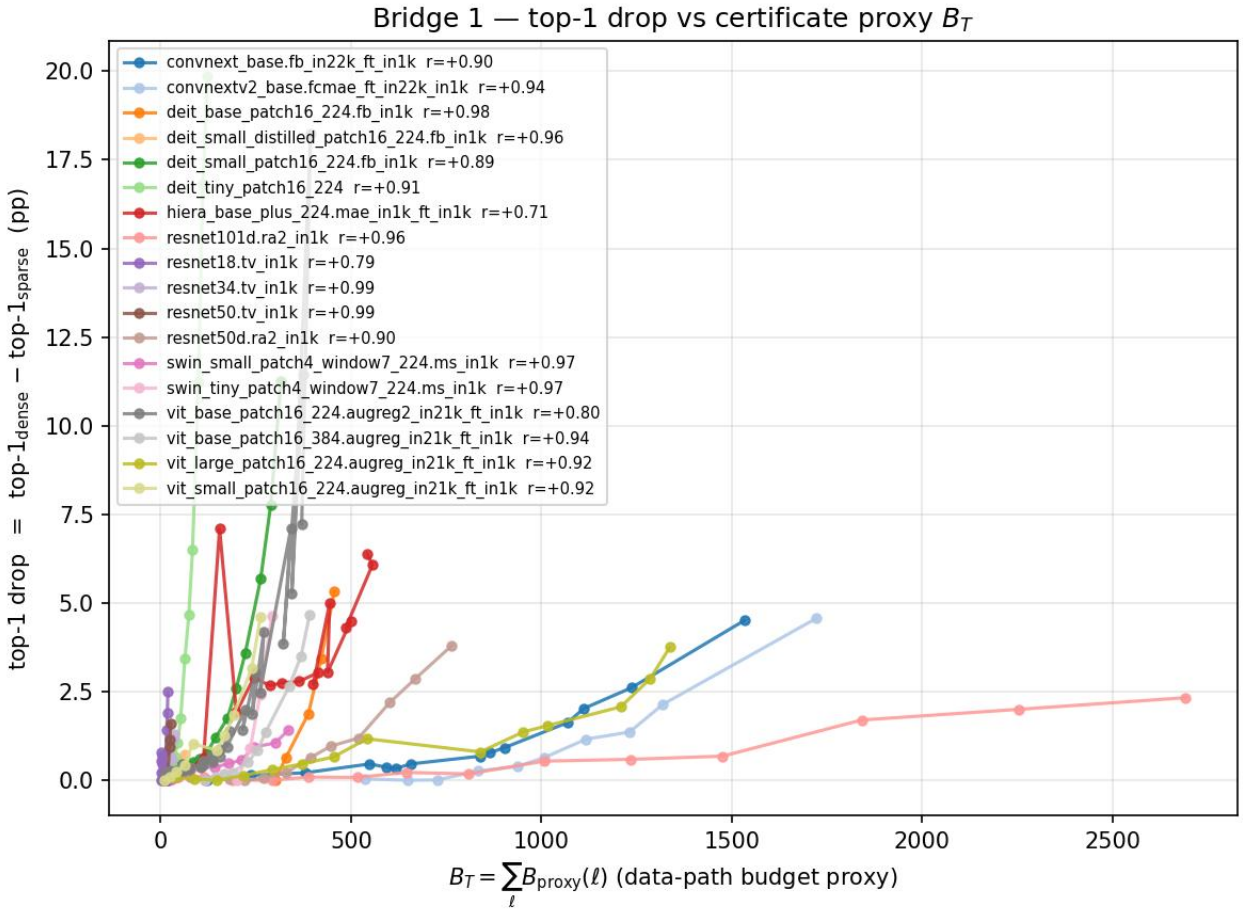


Figure 7: **Top-1 drop correlates with the operator-norm proxy $\hat{B}_T$ across all 18 paper architectures.** Each curve is one architecture; each marker is one Hybrid Magnitude–SER cycle. Within-architecture Pearson correlation between the operator-norm proxy $\hat{B}_T = \sum_\ell \|R_\ell\|_{\text{op}} \overline{\|\psi_{1,\ell}\|_\infty}$ and the post-fine-tuning top-1 drop is positive for every architecture: r ranges from +0.71 (Hiera-Base+) to +0.99 (ResNet50/34 `tv_in1k`), with mean +0.91 over the 18 models. We report this as an empirical correlation: the proxy $\hat{B}_T$ is not the deterministic budget $B_T(R)$ of Eq. (see the main paper) ($\|R\|_{\text{op}} \|\psi\|_\infty$ is not in general an upper bound for $\|R\psi\|_\infty$), so this correlation is suggestive of the certificate quantity's relevance but does not by itself verify Lemma the main paper.

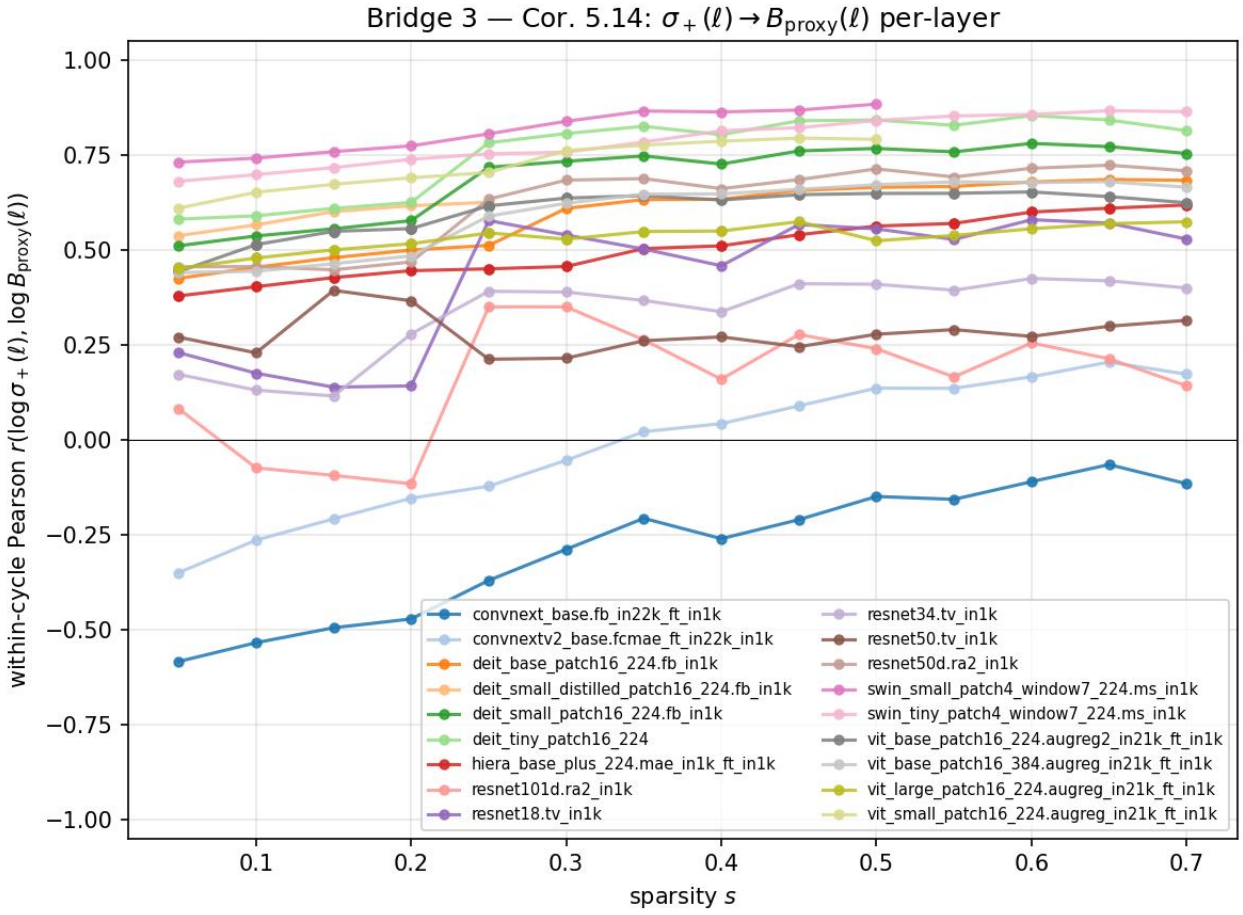


Figure 8: **Within-cycle log-log Pearson correlation $r(\log \sigma_+(\ell), \log \hat{B}_{T,\ell})$ as a function of sparsity s, per architecture.** Transformer architectures (Swin/DeiT/ViT) cluster in the +0.5 to +0.81 band, consistent with the iid-Gaussian heuristic of Corollary the main paper (which is a sufficient condition under iid weights, not a certificate for the realized magnitude-selected mask). ResNet/Hiera families are positive but weaker (+0.16 to +0.62). The two ConvNeXt rows are the only architectural exception: ConvNeXtV2-Base averages ≈ 0 and ConvNeXt-Base averages -0.29. Per-layer-type stratification shows the ConvNeXt anomaly is concentrated in depthwise-convolution layers; ConvNeXt MLP layers behave like ViT MLP layers ($r = +0.51$ and $+0.58$ respectively).

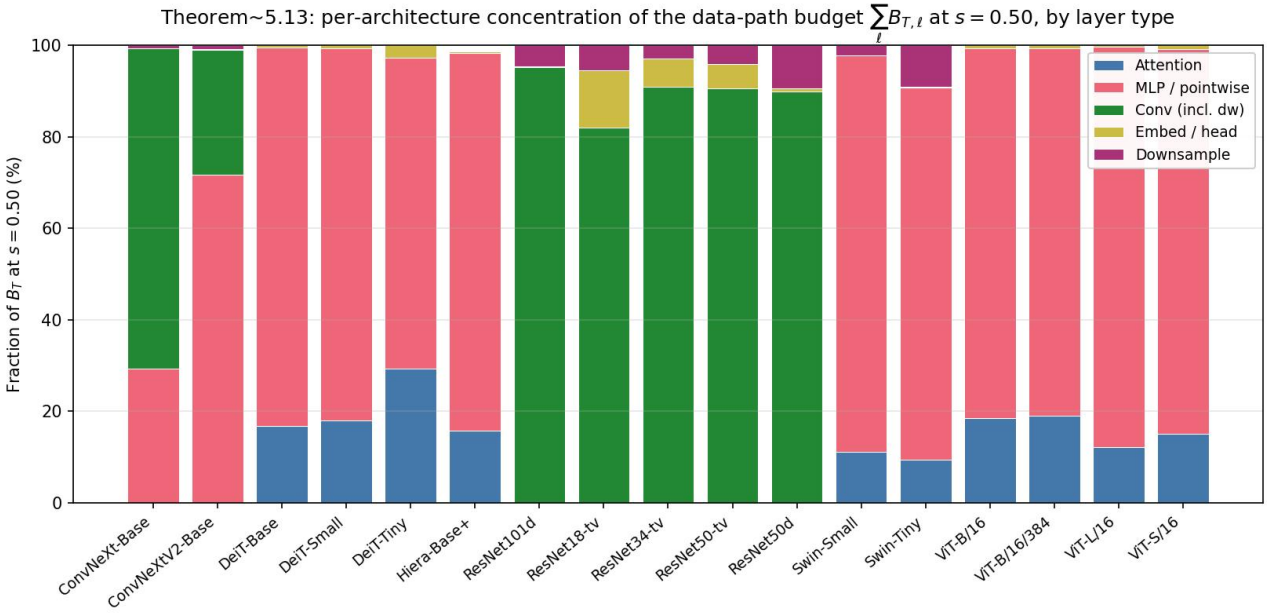


Figure 9: **Per-architecture decomposition of $\sum_\ell \hat{B}_{T,\ell}$ at $s = 0.50$ by layer type.** The bar height of each segment is its fraction of the total B_{total} for that architecture. Transformers (DeiT/Swin/ViT) are dominated by MLP/pointwise; ConvNeXt-Base is dominated by depthwise convolutions, ConvNeXtV2-Base by MLP/pointwise; ResNets by their convolutions.

Table 17: **Parameter-to-compute conversion: aggressive ToMe + 2:4 starting from a Hybrid Magnitude–SER $s = 0.35$ checkpoint.** Two FLOP methods are compared. The first row ("magnitude 2:4 + ToMe") preserves the existing unstructured sparse mask, then projects compatible Linear layers to a 2:4 pattern by keeping the two largest-magnitude entries per contiguous 4-tuple; the second row (**CAST**, certificate-aware 2:4 + ToMe + free restoration) replaces the magnitude rule with a row-wise discrete search that minimizes a Lemma the main paper-inspired activation-weighted cost over the 6 possible 2-of-4 patterns per output-channel and input-group, and additionally restores entries the unstructured mask had zeroed at zero sparse-execution FLOP cost (Theorem the main paper-style "free restoration" inside each 2:4 group). Both rows then apply ToMe $r = 8$ [3] and fine-tune for three epochs with the structured mask frozen. The CAST procedure is documented in detail in Appendix the main paper. The reported compute reduction is a sparse-execution MAC estimate: it assumes 2:4 Sparse Tensor Core-style execution for Linear GEMMs [20] and reports the ToMe-only dense-kernel reduction separately. We measure wall-clock throughput on an L4 GPU at batch 128 for two endpoints of the *same* ToMe-r=8 graph: $\sim$593 images/s when Linear layers run as dense GEMMs and $\sim$**837 images/s** when the 2:4-projected Linear weights are routed through PyTorch `to_sparse_semi_structured` kernels — a 1.41× measured speedup attributable to the 2:4 kernel switch alone, with ToMe held constant on both endpoints. The ToMe contribution over the un-tokened dense ViT-B is given only as the analytical $1/(1-0.2281) \approx 1.30\times$ MAC ratio; we do not report a measured ToMe-only-vs-original-dense throughput here.

Architecture	Source s	Method	Dense MACs	Sparse-exec red.	Top-1 (%)
ViT family (CAST = cert-aware 2:4 + free restoration + ToMe r=8 + 3-ep distill FT)					
ViT-B/16	0.35	Magnitude 2:4 + ToMe	17.56G	59.81%	82.92
ViT-B/16	0.35	**CAST**	17.56G	59.81%	**83.41**
ViT-B/16	0.35	**CAST 6:12 SER+α=0.5** (no ToMe)	17.56G	50%	**83.74**[§]
ViT-L/16	0.35	CAST	61.55G[†]	∼60%[†]	**84.37**
DeiT-B	0.35	CAST	17.56G	59.81%	80.48
DeiT-S	0.35	CAST	4.61G	59.81%	76.96
DeiT-T	0.35	CAST	1.26G	59.81%	65.93
ResNet family (CAST-conv = cert-aware 1×1+3×3 2:4 + free restoration; ToMe N/A)					
ResNet50	0.35	CAST-conv	4.09G	48.5%	73.14
ResNet50	0.35	**CAST-conv+perm**	4.09G	48.5%	**75.67**[‡]
ResNet50d	0.35	CAST-conv	4.33G	49.85%	78.08
ResNet50d	0.35	**CAST-conv+perm**	4.33G	49.85%	**78.00**[‡]
ResNet101d	0.35	CAST-conv	8.0G	∼50%	80.13
ResNet101d	0.35	**CAST-conv+perm**	8.0G	∼50%	**80.59**[‡]
ResNet152d	0.35	**CAST-conv+perm**	11.8G	∼50%	**81.33**[‡]
ConvNeXt family (CAST on the nn.Linear pointwise convs; ToMe N/A)					
ConvNeXt-Base	0.35	CAST	15.4G	TBD	TBD
ConvNeXtV2-Base	0.35	**CAST 2:4 cert + free-restore**	15.4G	50%	**85.47**
ConvNeXtV2-Base	0.35	**CAST 12:16 dense+perm** (25% sparse)	15.4G	25%	**86.35**[§]
ConvNeXtV2-Base	0.35	**CAST 8:16 dense+perm** (50% sparse)	15.4G	50%	**85.85**[§]

[†]ViT-L/16 reduction is estimated from the analytical 22.81% ToMe + 50% 2:4-Linear share. [‡]**CAST-conv+perm** = CAST-conv with the importance-balanced Cin permutation alignment of Appendix the main paper ("flatten the layer"), motivated by the ResNet pre-FT collapse paragraph below. ResNet152d row was added in this revision; CAST-conv+perm achieves 81.33% post-FT on `resnet152d.ra2_in1k` (dense baseline 82.86%), a −1.53 pp drop at ∼50% sparse-execution MAC reduction. [§]New rows added in this revision (May 9): (i) ViT-B/16 **6:12 SER+α=0.5 inline** — same 50% N:M structured sparsity as the canonical 2:4 row but at $n = 12$ (more pattern flexibility per group), no ToMe, dense-teacher distillation; this lands at 83.74% post-FT, **+0.33** pp above the canonical 2:4+ToMe row at the same parameter sparsity, demonstrating the n-flexibility benefit of wider N:M patterns. (ii) ConvNeXtV2-Base **12:16 dense+perm** — 25% sparsity (vs 50% for 2:4); reaches 86.35% post-FT, only −0.37 pp from dense 86.72%, the "minimal accuracy loss" density lane.

Table 18: **One-to-one mapping from CAST/CAST-conv+perm pipeline steps to the certificate theorems of Section the main paper.** Each row pairs an empirical knob in the implementation (left) with the specific theorem, lemma, or corollary that motivates it (right). The pipeline is not a heuristic stack: every component is a certificate-improving move under the deterministic mask-certificate framework, with the source SER checkpoint chosen by the Marchenko–Pastur edge analysis, the per-layer pruning budget allocated by Corollary the main paper, the 2:4 mask selected by minimising a Lemma the main paper-inspired cost, the prune–restore step certificate-improving by Theorem the main paper, the α_{ser} prior biasing toward the SER-validated kept set, and the Cin permutation alignment minimising the dispersion of B_T contributions across 4-tuples.

Pipeline step	Theorem / lemma in paper
SER $s = 0.35$ starting checkpoint	Marchenko–Pastur edge σ_+ as bulk/spike threshold (Cor. the main paper); the bulk-only pruning rule chooses entries below σ_+ and so is certificate-cheap by construction.
Per-layer pruning budget (SEB allocator)	Cor. the main paper — high-σ_+ layers are the random-capacity reservoirs; allocate more pruning there because $B_T(R_\ell)$ responds most weakly per unit removed Frobenius mass.
Cert-aware 2:4 mask (CAST core)	Lemma the main paper — pick the 2-of-4 keep pattern that minimises the activation-weighted removed-contribution cost $B_T(R)$ per output row, rather than sharing one pattern across rows.
Free restoration	Theorem the main paper — splitting the removed mass into a kept reservoir Q and a finally-removed component P; restoring Q at zero sparse-execution FLOP cost strictly improves the certificate margin when $B_T(P)$ exceeds the regularisation penalty $\lambda_2\|P\|_F^2 + \lambda_1\|P\|_1$.
α_{ser} Hamming prior	Theorem the main paper + an empirical Bayesian prior on the certificate-improving Q: bias the 2:4 keep pattern toward the SER-validated unstructured mask, since SER kept entries are bulk-edge-validated.
Permutation alignment (CAST-conv+perm)	Cin importance balancing minimises the dispersion of B_T contributions across 4-tuples; the importance-balanced quartile interleave of Appendix the main paper ("flatten the layer") directly reduces the max-per-group certificate cost.

Table 19: ViT-B/16 head-to-head reference at $\sim 30\%$ compression on ImageNet-1k validation. CP-ViT figures are extracted from Fig. 3 of the prior preprint; the comparison is indicative because of the checkpoint mismatch (CP-ViT baseline 77.91% vs. our 85.11%). Hybrid Magnitude–SER and the three within-paper baselines are reproduced from Table the main paper. All entries are post-fine-tuning unless noted; CP-ViT used 30 FT epochs vs. our cumulative ~ 6 FT epochs through $s = 0.30$.

Method	Top-1 (%)	Δ (pp)	Compression
ViT-B/16 [7] (our dense baseline 85.11; CP-ViT baseline 77.91)			
CP-ViT [25], no FT*	~ 76.4	~ -1.5	$\sim 30\%$ FLOPs
CP-ViT [25], with FT*	~ 77.0	~ -0.9	$\sim 30\%$ FLOPs
Classical magnitude (this paper)	84.46	-0.65	30% param
Classical RMT (this paper)	84.55	-0.56	30% param
SER (this paper)	84.55	-0.56	30% param
Hybrid Magnitude–SER (this paper)	**84.64**	**-0.47**	**30%** param
ViT-B/16 at higher compression (only this paper has direct numbers)			
Classical magnitude, $s = 0.50$	81.67	-3.44	50% param
Classical RMT, $s = 0.50$	82.57	-2.54	50% param
SER, $s = 0.50$	83.27	-1.84	50% param
Hybrid Magnitude–SER, $s = 0.50$	**83.37**	**-1.74**	**50%** param
ViT-B/16 at high compression (only this paper has direct numbers)			
Classical magnitude, $s = 0.65$	74.30	-10.81	65% param
Classical RMT, $s = 0.65$	76.46	-8.65	65% param
SER, $s = 0.65$	73.80	-11.31	65% param
Hybrid Magnitude–SER, $s = 0.65$	**79.95**	**-5.16**	**65%** param
Hybrid Magnitude–SER, $s = 0.70$	**78.01**	**-7.10**	**70%** param
Classical magnitude, $s = 0.70$	67.44	-17.67	70% param
Classical RMT, $s = 0.70$	71.53	-13.58	70% param

*Approximate values read from Fig. 3 of the prior preprint; CP-ViT used a different (77.91%) ViT-B/16 checkpoint. Numerical entries for our methods are from Table the main paper.

Table 20: Head-to-head DeiT pruning comparison after fine-tuning. "Top-1" is the post-pruning ImageNet-1k validation accuracy and "Δ" is the absolute change from the dense baseline used by that method. "Compression" is FLOPs savings for VTP/PoWER/HVT/CP-ViT and parameter sparsity for Hybrid Magnitude–SER (see protocol caveats above). VTP, PoWER, and HVT DeiT numbers are reproduced from CP-ViT [25]. Hybrid Magnitude–SER entries are included only where an archived full-validation checkpoint is available.

Method	Top-1 (%)	Δ (pp)	Compression
DeiT-Tiny 16/224 [28] (prior dense 72.20; our dense 72.21)			
VTP [34]†	70.55	−1.65	45.32% FLOPs
PoWER-BERT [10]†	70.05	−2.15	41.26% FLOPs
HVT [21]†	70.01	−2.19	47.32% FLOPs
CP-ViT [25]	71.24	−0.96	43.34% FLOPs
Hybrid Magnitude–SER, $s = 0.20$ **(this paper)**	**71.86**	**−0.35**	**20%** param
DeiT-Small 16/224 [28] (prior dense 79.80; our dense 79.85)			
VTP [34]†	78.24	−1.56	42.52% FLOPs
PoWER-BERT [10]†	78.30	−1.50	41.36% FLOPs
HVT [21]†	78.05	−1.75	47.80% FLOPs
CP-ViT [25]	79.08	−0.72	42.24% FLOPs
Hybrid Magnitude–SER, $s = 0.20$ **(this paper)**	**79.37**	**−0.48**	**20%** param
Hybrid Magnitude–SER, $s = 0.40$ **(this paper)**	78.61	−1.24	40% param
DeiT-Base 16/224 [28] (prior dense 81.82; our dense 81.80)			
VTP [34]†	80.70	−1.12	43.20% FLOPs
PoWER-BERT [10]†	80.17	−1.65	39.24% FLOPs
HVT [21]†	79.94	−1.88	44.78% FLOPs
CP-ViT [25]	81.13	−0.69	41.62% FLOPs
Hybrid Magnitude–SER, $s = 0.20$ **(this paper)**	**81.46**	**−0.34**	**20%** param
Hybrid Magnitude–SER, $s = 0.35$ **(this paper)**	81.17	−0.63	35% param
Hybrid Magnitude–SER, $s = 0.40$ **(this paper)**	80.99	−0.81	40% param

†Reproduced by CP-ViT [25]; not reported in the original VTP/PoWER/HVT papers.